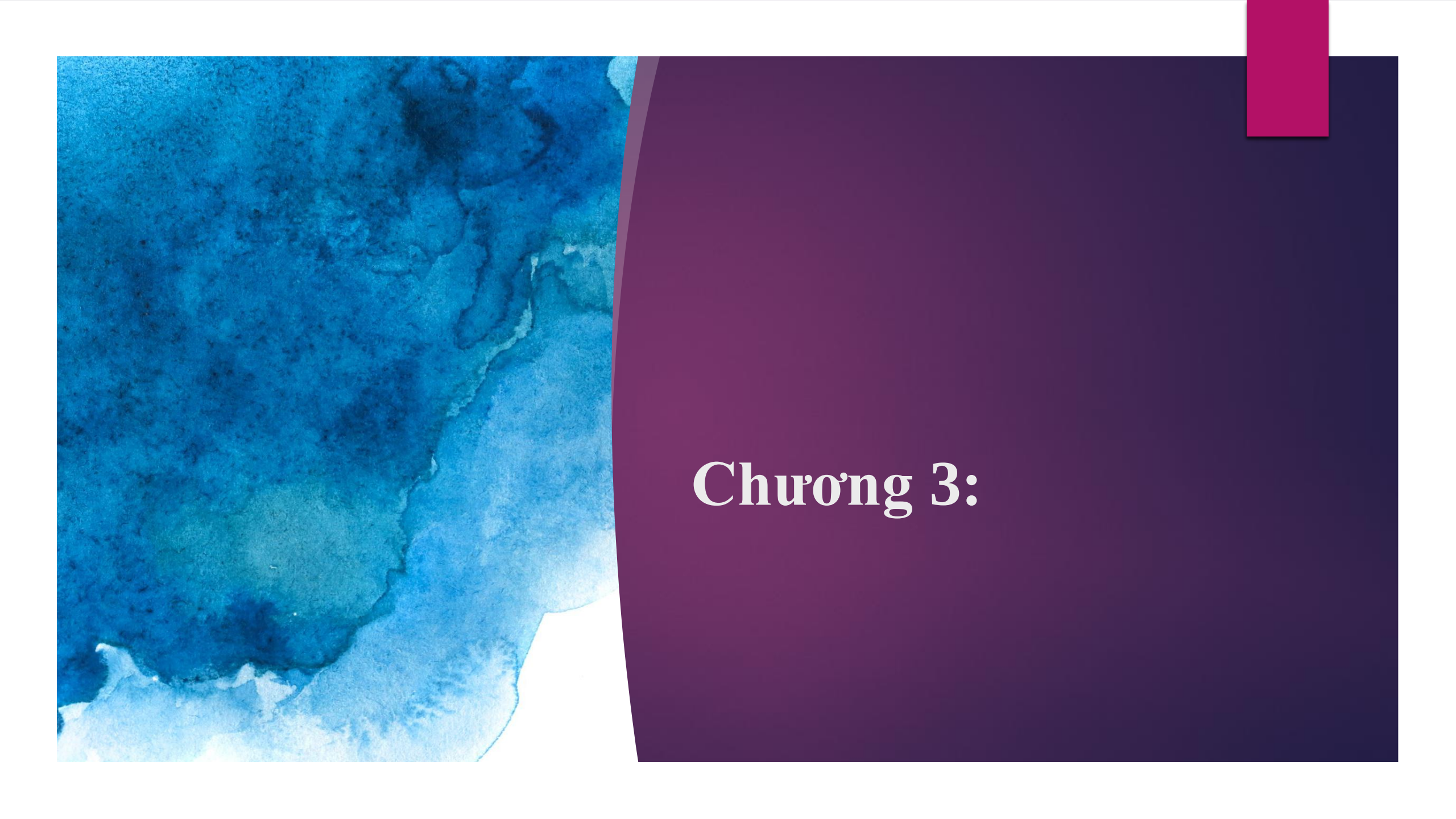




Chương 3:

Mục lục

Giới thiệu sơ lược

1. Mã hóa entropy
2. Mã hóa dự đoán
3. Dự đoán bù chuyển động
4. Lượng tử hóa

1. Mã hóa entropy

- ▶ Ý tưởng sử dụng mã hóa độ dài thay đổi để biểu diễn thông điệp một cách hiệu quả được gọi là mã hóa entropy.
- ▶ Kỹ thuật tốt nhất và được sử dụng rộng rãi nhất được gọi là mã hóa Huffman.
- ▶ Các tiêu chuẩn nén video và âm thanh hiện tại được sử dụng cho truyền hình kỹ thuật số bao gồm các từ mã độ dài thay đổi được tạo ra dưới dạng mã Huffman.

1.1.Mã Huffman

- ▶ Các bước liên quan đến việc tạo ra một bộ từ mã Huffman:
 - Bước 1: Liệt kê các ký hiệu cần truyền theo thứ tự xác suất giảm dần.
 - Bước 2: Kết hợp hai ký hiệu có xác suất nhỏ nhất và sắp xếp lại các ký hiệu theo thứ tự xác suất giảm dần.
 - Bước 3: Lặp lại Bước 2 cho đến khi chỉ còn lại hai ký hiệu.

1.1. Mã Huffman

- Bước 4: Đánh dấu mỗi kết hợp một cách nhất quán với:

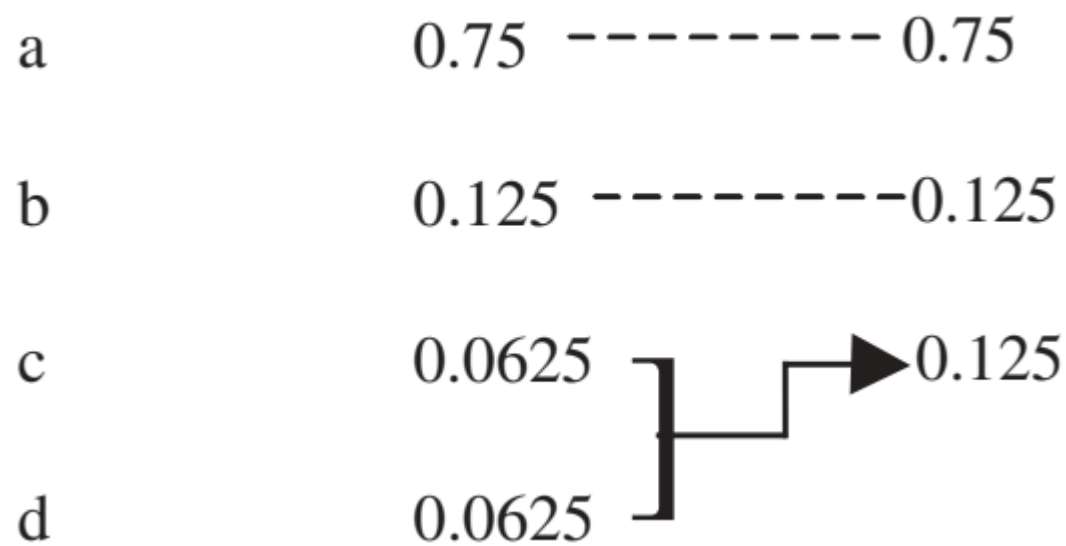
$$\begin{matrix} 1 \\ \text{J} \\ 0 \end{matrix} \quad \text{or} \quad \begin{matrix} 0 \\ \text{J} \\ 1 \end{matrix}$$

- Bước 5: Đọc các từ mã Huffman từ phải sang trái.

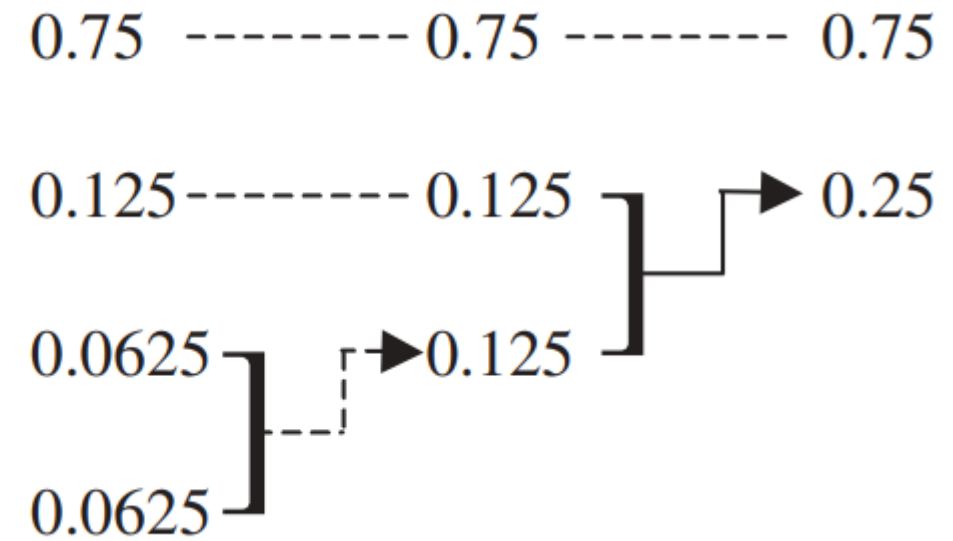
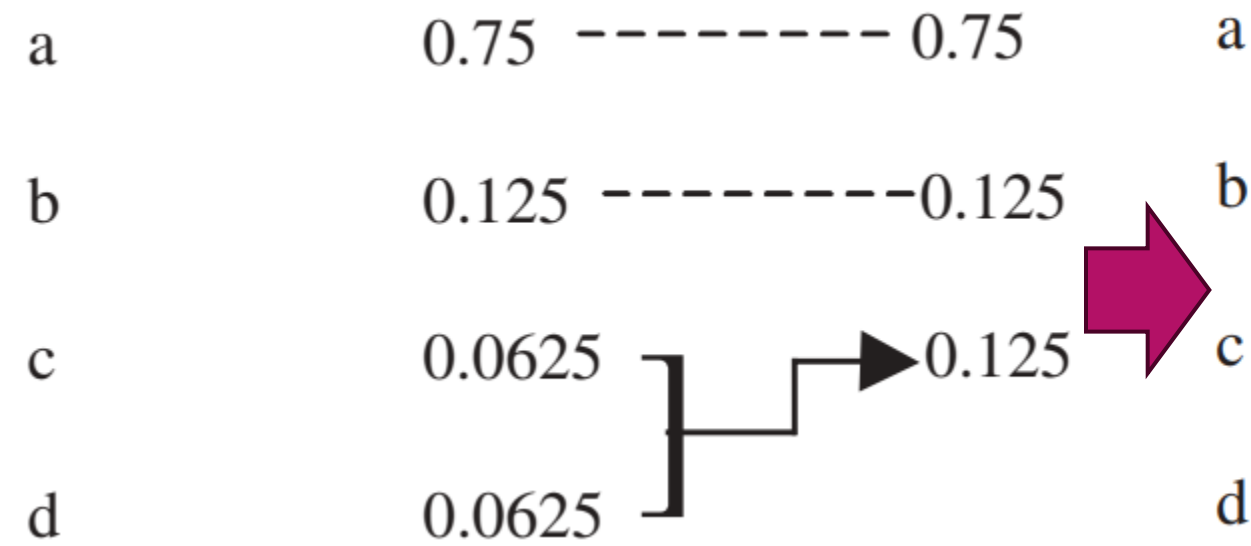
1.1.Mã Huffman

- Ví dụ: Các ký hiệu a, b, c và d có xác suất lần lượt là 0,75, 0,125, 0,0625 và 0,0625. Tính toán từ mã Huffman phù hợp cho các ký hiệu này.

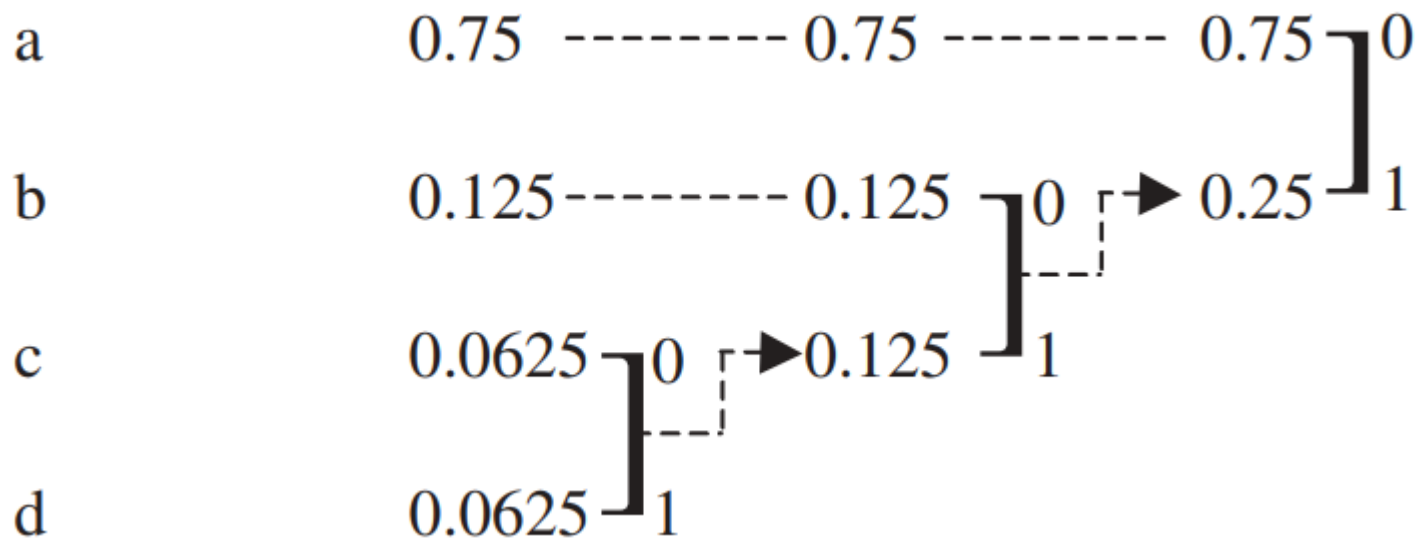
a	0.75
b	0.125
c	0.0625
d	0.0625



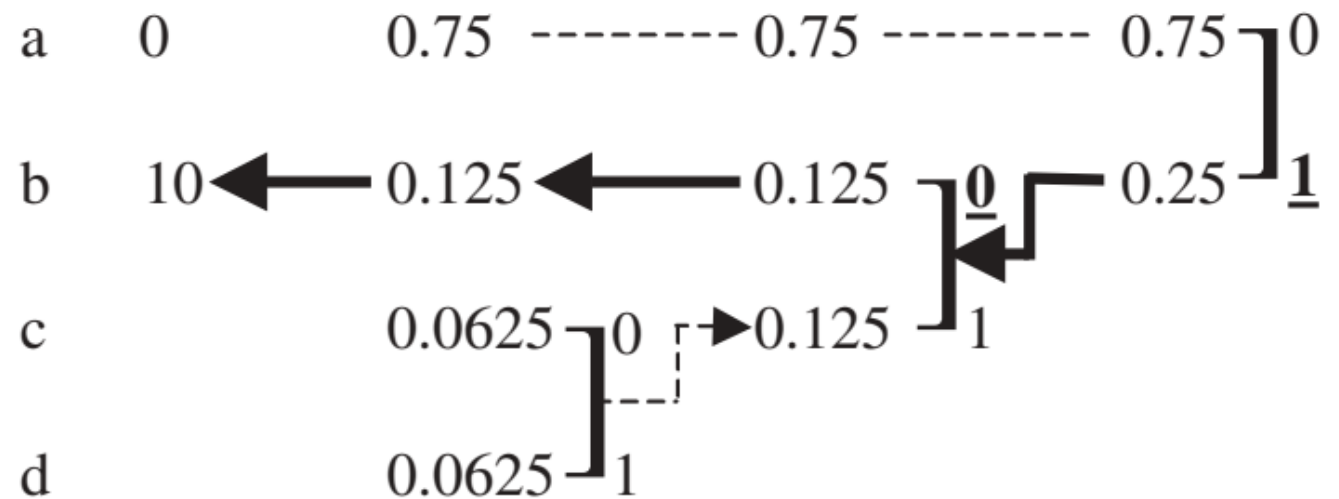
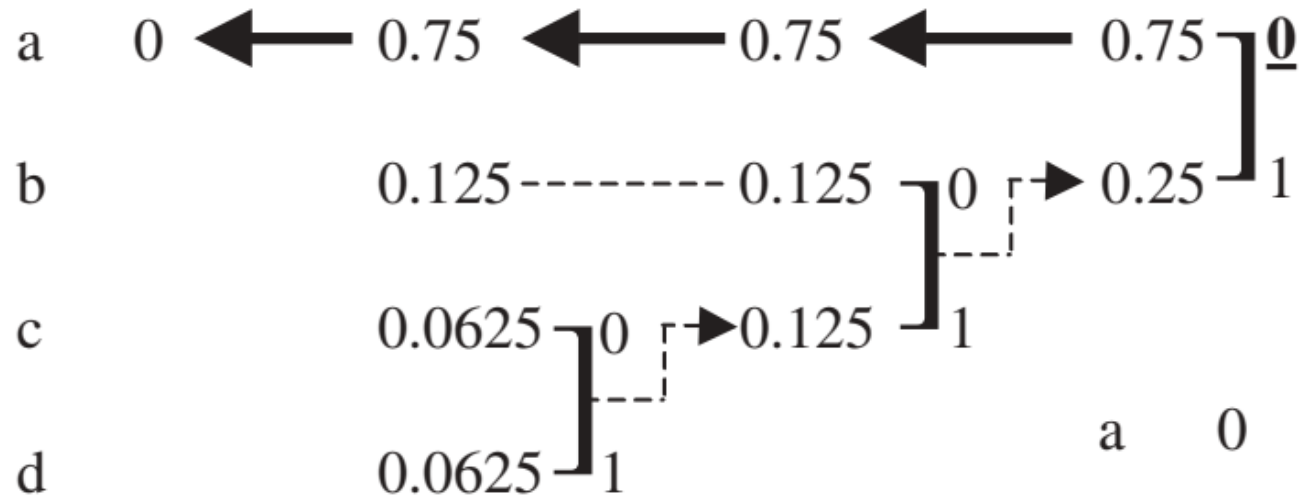
1.1. Mã Huffman



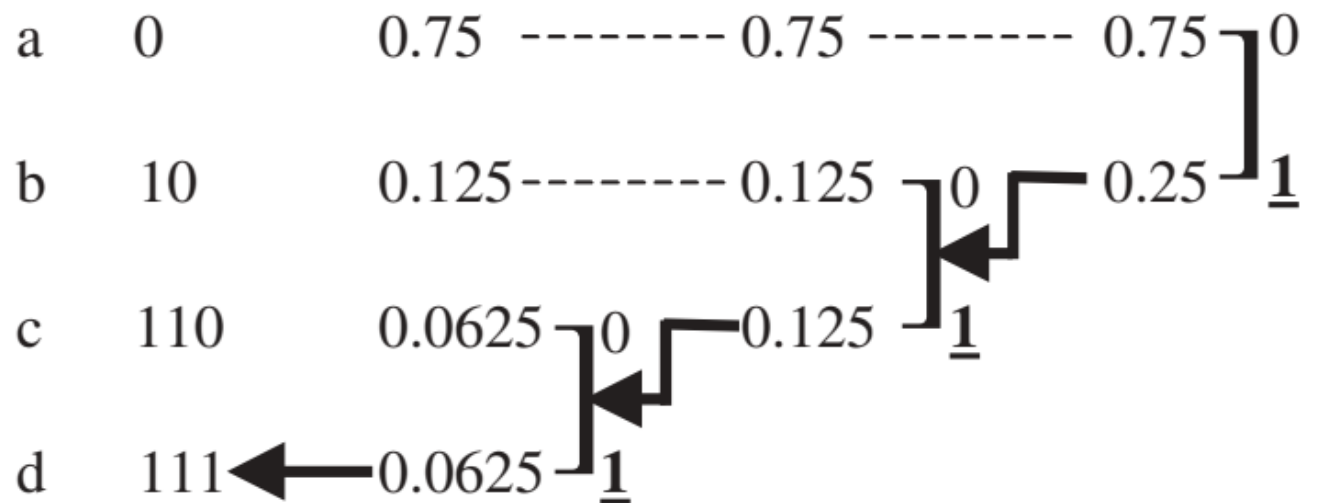
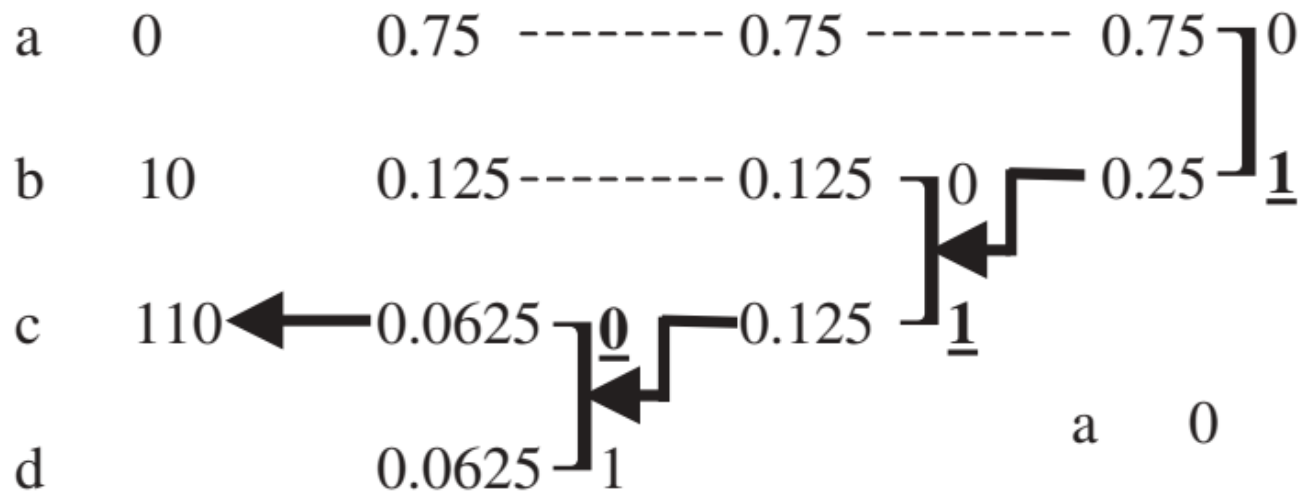
1.1.Mã Huffman



1.1.Mã Huffman



1.1.Mã Huffman



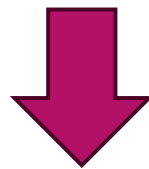
1.1.Mã Huffman

Symbol	Huffman code word
a	0
b	10
c	110
d	111

1.1.Mã Huffman

- Ví dụ: Sử dụng mã Huffman ở ví dụ trên để mã hóa luồng ký hiệu: a b c d a a a b

0 10 110 111 0 0 0 10



01011011100010

1.1. Mã Huffman

- ▶ Quá trình giải mã bao gồm việc đọc các giá trị từ luồng bit cho đến khi phát hiện ra một từ mã hợp lệ.
- ▶ Vì không có từ mã Huffman nào tạo thành từ tiền tố của một từ mã Huffman dài hơn, nên điều này luôn dẫn đến giải mã đúng trong môi trường không có lỗi.

1.1.Mã Huffman

0 1011011100010

a

0 1 011011100010

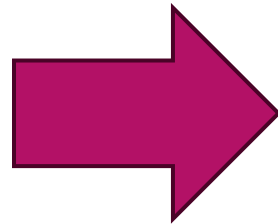
a ?

0 10 11011100010

a b

0 10 1 1011100010

a b ?



0 10 11 011100010

a b ?

0 10 110 11100010

a b c

0 10 110 1 1100010

a b c ?

0 10 110 11 100010

a b c ?

1.1.Mã Huffman

0	10	110	111	00010			
a	b	c	d				
0	10	110	111	0	0010		
a	b	c	d	a			
0	10	110	111	0	0	010	
a	b	c	d	a	a		
0	10	110	111	0	0	0	10
a	b	c	d	a	a	a	

1.1.Mã Huffman

0	10	110	111	0	0	0	1	0
a	b	c	d	a	a	a	?	
0	10	110	111	0	0	0	10	
a	b	c	d	a	a	a	b	

1.1. Mã Huffman

- ▶ Ví dụ: Luồng bit được tạo ra ở ví dụ trên có một lỗi bit duy nhất ở bit thứ hai của luồng bit. 00011011100010

Xác định trình tự được giải mã trong trường hợp này

1.1.Mã Huffman

0 0011011100010

a

0 0 011011100010

a a

0 0 0 11011100010

a a a

0 0 0 1 1011100010

a a a ?

1.1. Mã Huffman

0	0	0	11	011100010	
a	a	a	?		
0	0	0	110	11100010	
a	a	a	c		
0	0	0	110	1	1100010
a	a	a	c	?	
0	0	0	110	11	100010
a	a	a	c	?	

1.1.Mã Huffman

0	0	0	110	111	00010			
a	a	a	c	d				
0	0	0	110	111	0	0010		
a	a	a	c	d	a			
0	0	0	110	111	0	0	010	
a	a	a	c	d	a	a		
0	0	0	110	111	0	0	0	10
a	a	a	c	d	a	a	a	

1.1. Mã Huffman

0	0	0	110	111	0	0	0	1	0
a	a	a	c	d	a	a	a	?	
0	0	0	110	111	0	0	0	10	
a	a	a	c	d	a	a	a	b	

1.1.Mã Huffman

- ▶ Nén đạt được bằng cách sử dụng mã hóa Huffman với cái giá phải trả là tính dễ bị lỗi của luồng bit được tạo ra.
- ▶ Các bộ mã hóa video hiện đại được thiết kế để giảm thiểu tác động của khuyết điểm này.
- ▶ Tuy nhiên, không thể loại bỏ hoàn toàn khuyết điểm này.

1.2. Mã Run-length

- ▶ Trong mã hóa run length, một chuỗi các ký hiệu giống hệt nhau liên tiếp được kết hợp với nhau và được biểu diễn bằng một từ mã có độ dài biến đổi duy nhất (ví dụ: Huffman).
- ▶ Một ví dụ đơn giản là việc truyền bản fax đen trắng.

100

-
- 3 2 5 1 2 6 3 3 3



1.2. Mã Run-length

- ▶ Trong trường hợp truyền fax, các bộ từ mã Huffman riêng biệt sẽ được tạo ra cho các lần chạy pixel trắng và các lần chạy pixel đen.
- ▶ Cách này được sử dụng vì số liệu thống kê cho các loại chạy khác nhau không giống nhau.

2. Mã hóa dự đoán

- ▶ Trong Chương 2, chúng ta đã thấy rằng việc biết giá trị của một pixel cụ thể cung cấp một lượng thông tin đáng kể về các pixel xung quanh nó do tương quan cao thường tồn tại giữa các điểm ảnh gần nhau.
- ▶ Điều cần thiết là tìm cách để khai thác mối tương quan này để có thể giảm lượng thông tin cần truyền đi.

2. Mã hóa dự đoán

- ▶ Một cách để đạt được là thông qua một kỹ thuật được gọi là mã hóa dự đoán (predictive code).
- ▶ Mục đích của mã hóa dự đoán là sử dụng các giá trị của các pixel đã truyền trước đó để dự đoán giá trị của pixel hiện tại đang được truyền.

2. Mã hóa dự đoán

- ▶ Sau đó, chỉ cần truyền sự khác biệt giữa dự đoán và giá trị pixel thực tế.
- ▶ Nếu dự đoán chính xác, thì đây thường là một giá trị nhỏ và do đó sẽ cần ít bit dữ liệu hơn để biểu diễn nó.
- ▶ Dạng mã hóa dự đoán đơn giản nhất là dự đoán giá trị của điểm ảnh được truyền bằng cách sử dụng giá trị của điểm ảnh ngay bên trái của nó.

2. Mã hóa dự đoán

- ▶ Cách tiếp cận này thường được gọi là điều chế xung mã vi sai một chiều (DPCM- Differential Pulse Code Modulation).
- ▶ Sau đó, sự khác biệt giữa dự đoán và giá trị điểm ảnh thực tế được truyền đi.

2. Mã hóa dự đoán

► X là điểm ảnh được truyền và A là giá trị của điểm ảnh ngay bên trái của nó.

► Lấy giá trị dự đoán của X là A

$$\tilde{X} = A$$

► Giá trị được truyền đi là:

$$Difference = X - \tilde{X} = X - A$$

A	X	

2. Mã hóa dự đoán

- Ví dụ: Cho các điểm ảnh sau. Tính toán các giá trị dự đoán khi dùng DPCM một chiều. Giả sử pixel đầu tiên trong một dòng được dự đoán bởi giá trị xám là 128.

130	135	141	129	151
124	140	165	200	199
119	132	175	205	203

2. Mã hóa dự đoán

Pixel value	130	135	141	129	151
Prediction	128	130	135	141	129
Pixel value	124	140	165	200	199
Prediction	128	124	140	165	200
Pixel value	119	132	175	205	203
Prediction	128	119	132	175	205

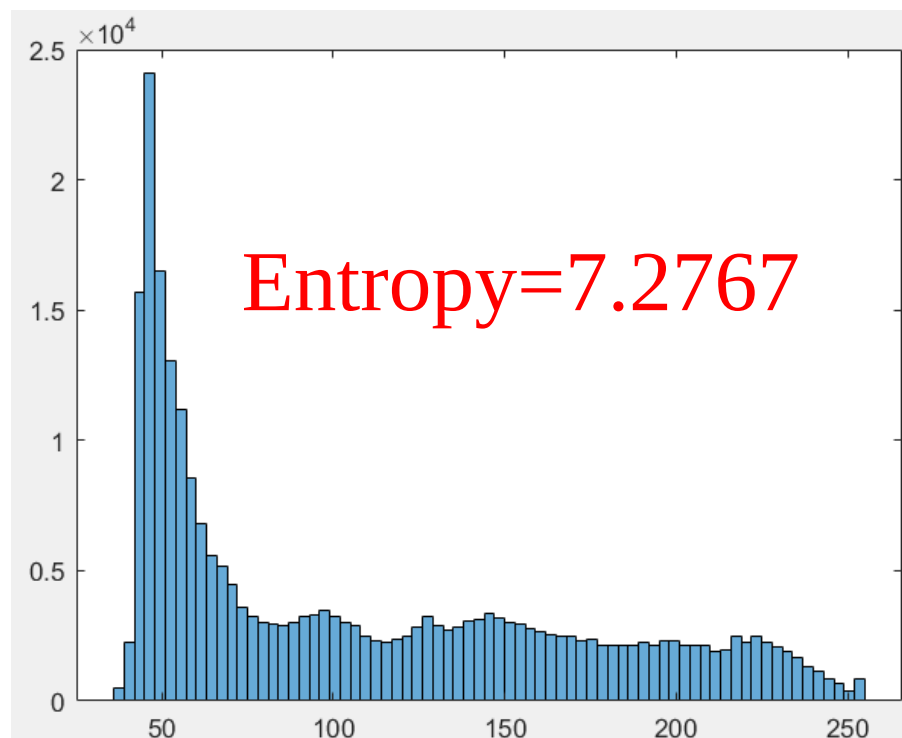
2. Mã hóa dự đoán

- Sự khác biệt sau đó được tính toán bằng cách trừ giá trị dự đoán khỏi giá trị pixel thực.

+2	+5	+6	-12	+22
-4	+16	+25	+35	-1
-9	+13	+43	+30	-2

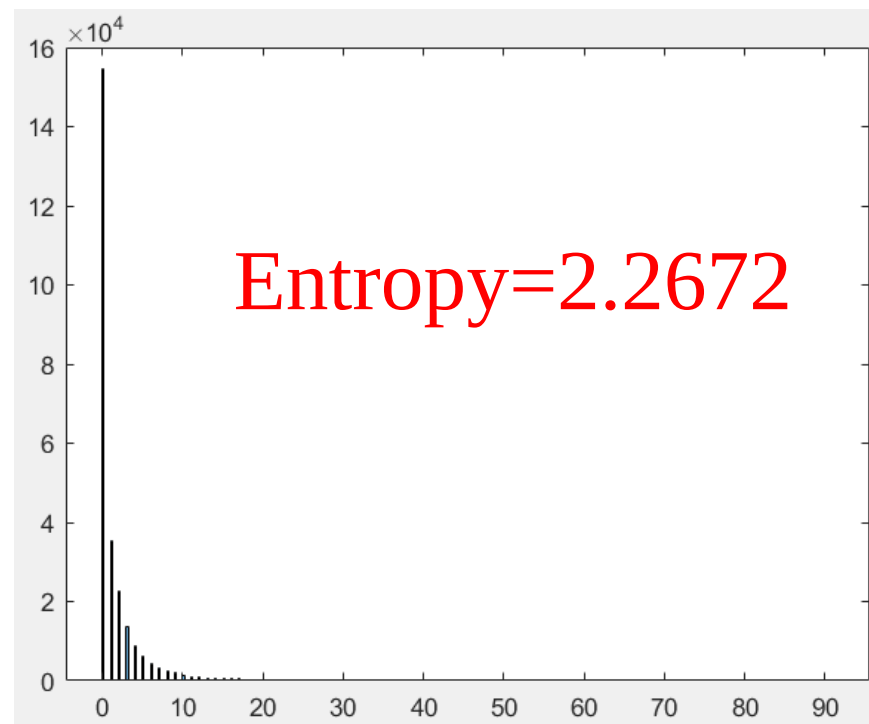
2. Mã hóa dự đoán

► Ví dụ: (Chạy file Matlab b3_01)



2. Mã hóa dự đoán

► Ví dụ: (Chạy file Matlab b3_02)



2. Mã hóa dự đoán

- ▶ Trong ví dụ trước, pixel bên trái của pixel được truyền đi được sử dụng làm bộ dự đoán.
- ▶ Tất nhiên, bất kỳ pixel nào có giá trị đã được truyền đến máy thu đều có thể được sử dụng.
- ▶ Ví dụ, pixel ngay phía trên của pixel được truyền đi có thể được sử dụng để hình thành dự đoán.

2. Mã hóa dự đoán

► Bộ dự đoán hai chiều tổng quát như hình vẽ

► Trong trường hợp này, dự đoán của X

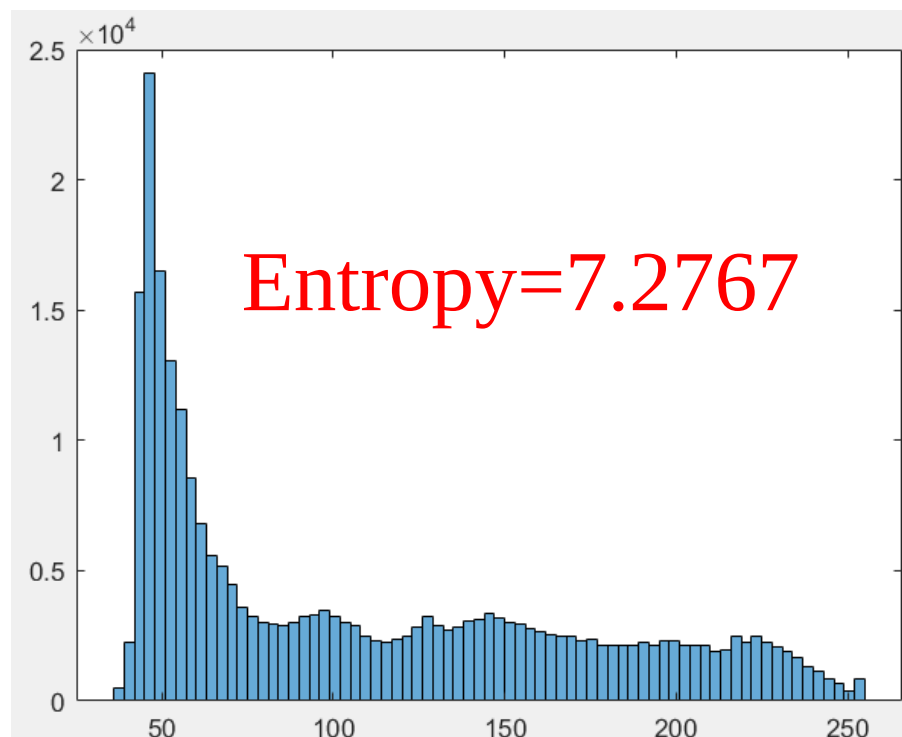
$$\tilde{X} = k_1A + k_2B + k_3C + k_4D$$

► Với k_i phụ thuộc vào hình ảnh

B	C	D
A	X	

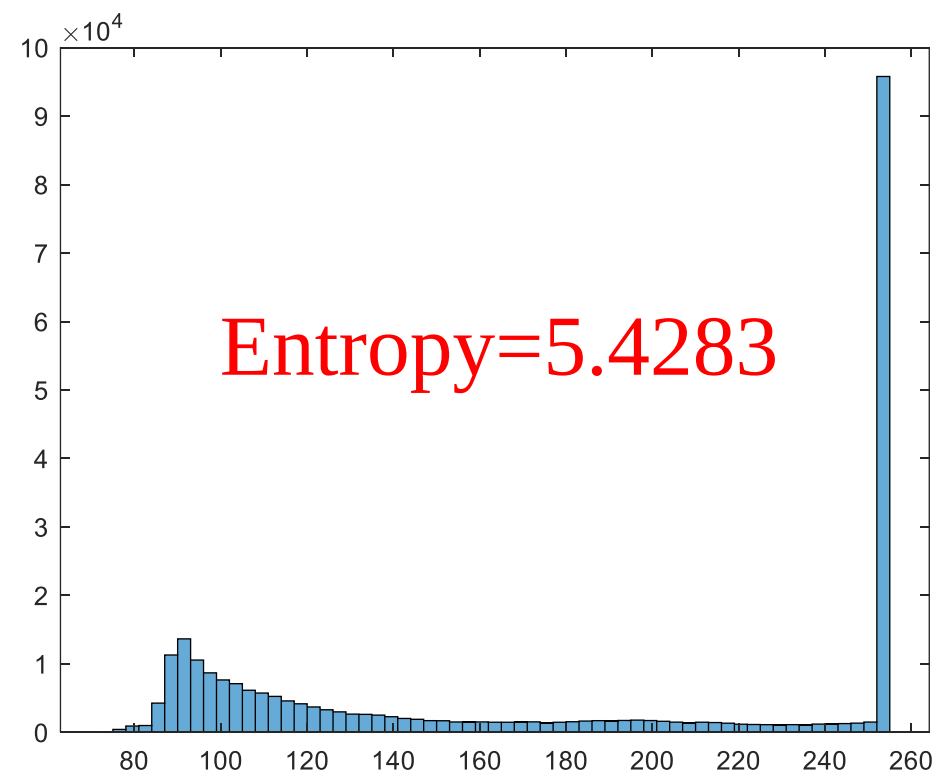
2. Mã hóa dự đoán

► Ví dụ: dự đoán dùng công thức sau: $\tilde{X} = 0.5A + 0.5C$



2. Mã hóa dự đoán

► Ví dụ: (Chạy file Matlab b3_03)



2. Mã hóa dự đoán

- ▶ Các nhà nghiên cứu đã chỉ ra rằng một khi các điểm ảnh A và C đã được sử dụng, thì việc sử dụng thêm các điểm ảnh trong cùng một hình ảnh sẽ không mang lại nhiều lợi ích.
- ▶ Bộ mã hóa dự đoán trong đó tất cả các điểm ảnh được sử dụng để tạo dự đoán đều nằm trong cùng một hình ảnh với điểm ảnh cần dự đoán được gọi là bộ mã hóa dự đoán trong hình ảnh (intrapicture predictive encoder)

2. Mã hóa dự đoán

- ▶ Cũng như dự đoán từ các pixel khác trong hình ảnh hiện tại, cũng có thể dự đoán từ một hoặc nhiều hình ảnh trước đó.
- ▶ Nếu có ít chuyển động giữa các hình ảnh thì dự đoán có thể hiệu quả.
- ▶ Điều này được gọi là dự đoán liên hình ảnh (**interpicture prediction**)

2. Mã hóa dự đoán

► Ví dụ: sử dụng dự đoán từ ảnh trước đó:

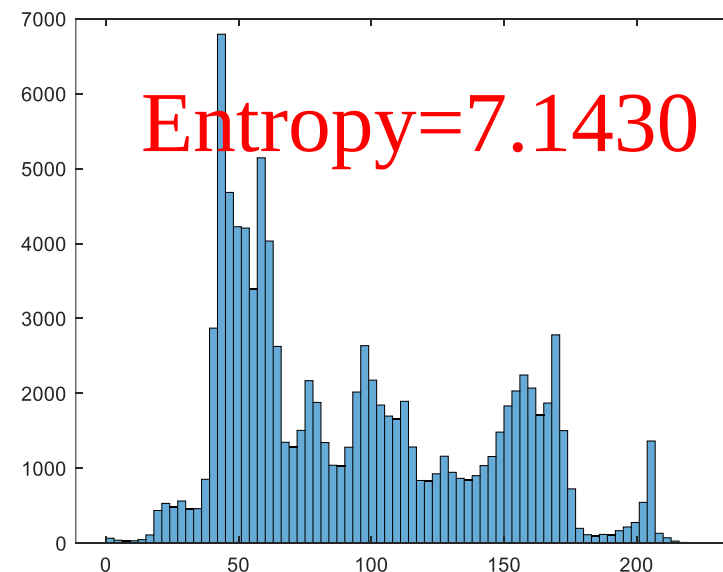
$difference = ảnh\ sau - ảnh\ trước$ (Chạy file Matlab b3_04)



Frame 29



Frame 30



Histogram
của frame
30

2. Mã hóa dự đoán

► Ví dụ: sử dụng dự đoán từ ảnh trước đó:

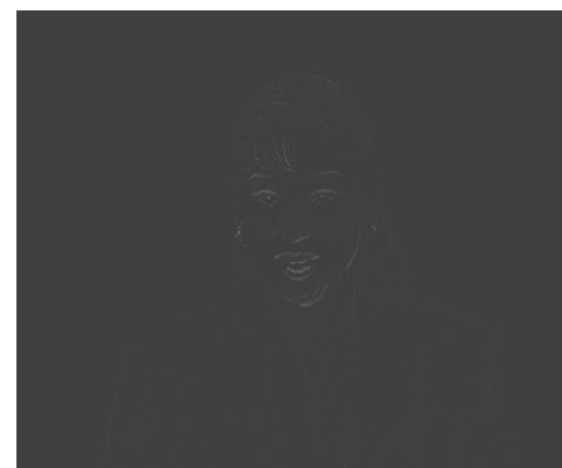
$difference = ảnh\ sau - ảnh\ trước$ (Chạy file Matlab b3_04)



Frame 29



Frame 30



Difference

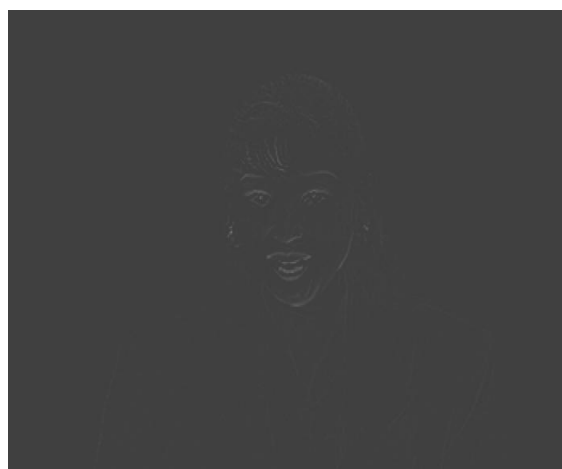
2. Mã hóa dự đoán

► Ví dụ: sử dụng dự đoán từ ảnh trước đó:

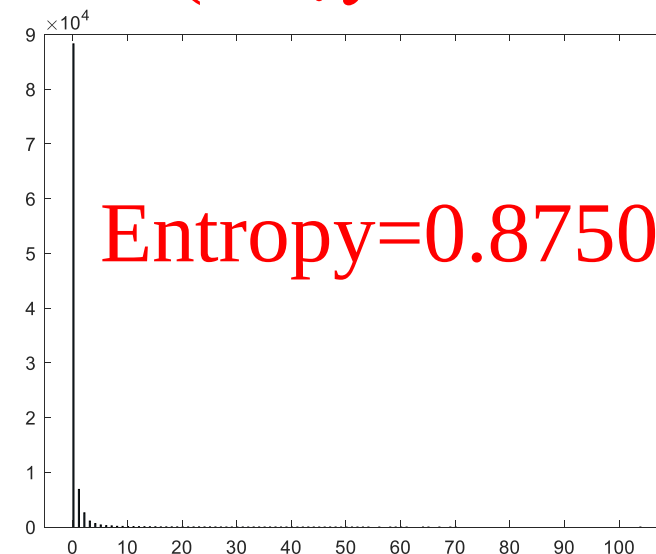
$difference = ảnh\ sau - ảnh\ trước$ (Chạy file Matlab b3_04)



Frame 30



Difference



Histogram
của
Difference

2. Mã hóa dự đoán

- ▶ Ví dụ: tạo video chuyển động từ 5 frame
(Chạy file Matlab b3_05)

3. Dự đoán bù chuyển động

- ▶ Dự đoán liên hình ảnh sẽ được cải thiện đáng kể nếu chúng ta có thể tính đến chuyển động xảy ra giữa các hình ảnh.
- ▶ Thành công có nghĩa là thay vì các đối tượng di chuyển cần được mã hóa hai lần trong một hình ảnh, chúng ta có thể loại bỏ yêu cầu mã hóa chúng hoàn toàn.
- ▶ Thay vào đó, thông tin về cách đối tượng di chuyển giữa các hình ảnh sẽ được truyền đi.

3. Dự đoán bù chuyển động

- ▶ Mặc dù điều này nghe có vẻ ổn về mặt lý thuyết, nhưng việc ước tính chuyển động trong một chuỗi hình ảnh lại phức tạp.
- ▶ Mỗi loại chuyển động ảnh hưởng đến sự xuất hiện theo thời gian của chuỗi video kết quả theo một cách cụ thể.
- ▶ Cố gắng triển khai các mô hình này trong phần cứng thời gian thực sẽ vô cùng phức tạp.

3. Dự đoán bù chuyển động

- ▶ May mắn thay, việc xấp xỉ mọi chuyển động như chuyển động tịnh tiến trong một mặt phẳng vuông góc với trục máy ảnh là một cách tiếp cận thỏa đáng với điều kiện là điều này được thực hiện trên các vật thể đủ nhỏ.
- ▶ Phần đầu tiên của quá trình này là cố gắng ước tính chuyển động của vật thể. Ước tính chuyển động được thực hiện bằng cách tìm kiếm hình ảnh dự đoán để tìm ra sự khớp nhất với vật thể quan tâm trong hình ảnh hiện tại cần mã hóa.

3. Dự đoán bù chuyển động

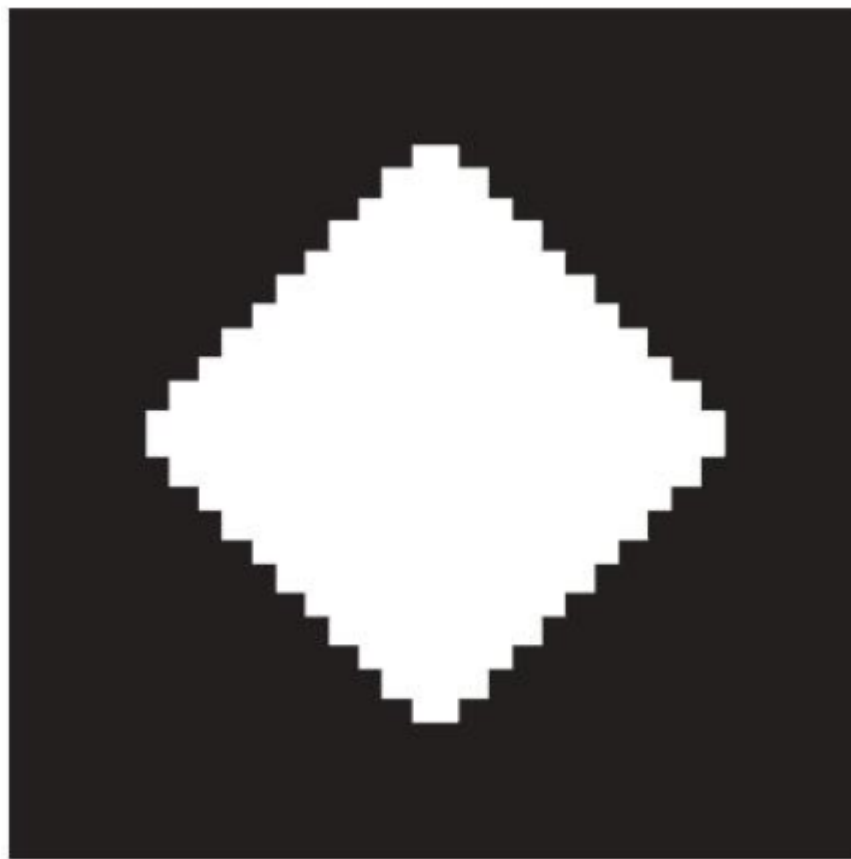


Hình ảnh hiện tại



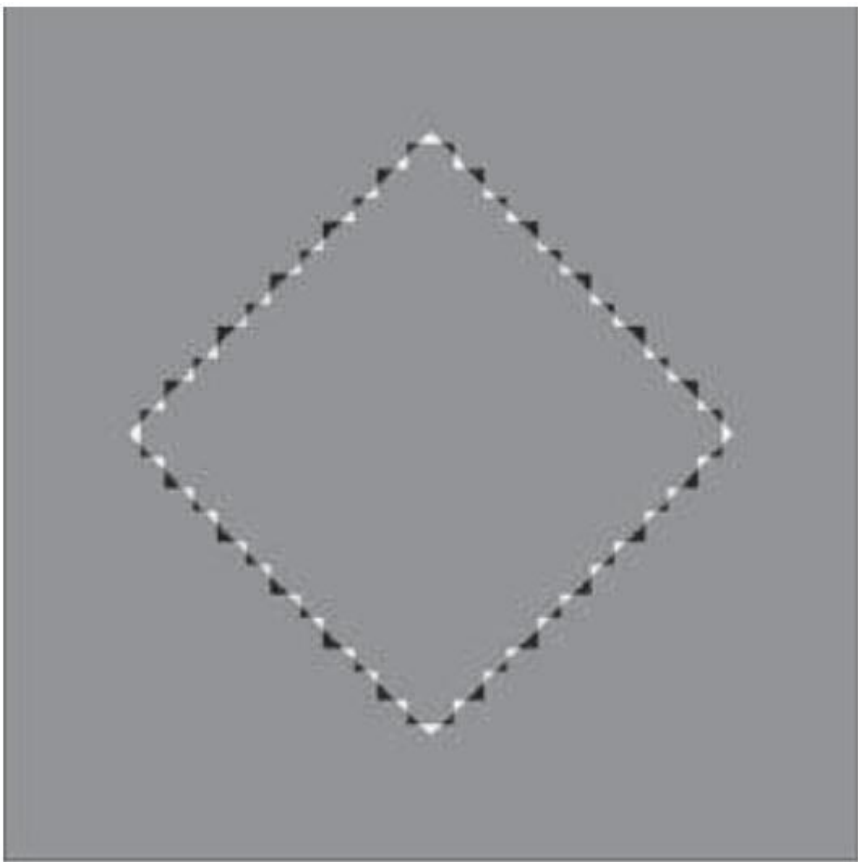
Hình ảnh dự đoán

3. Dự đoán bù chuyển động



Hình ảnh dự đoán bù chuyển động

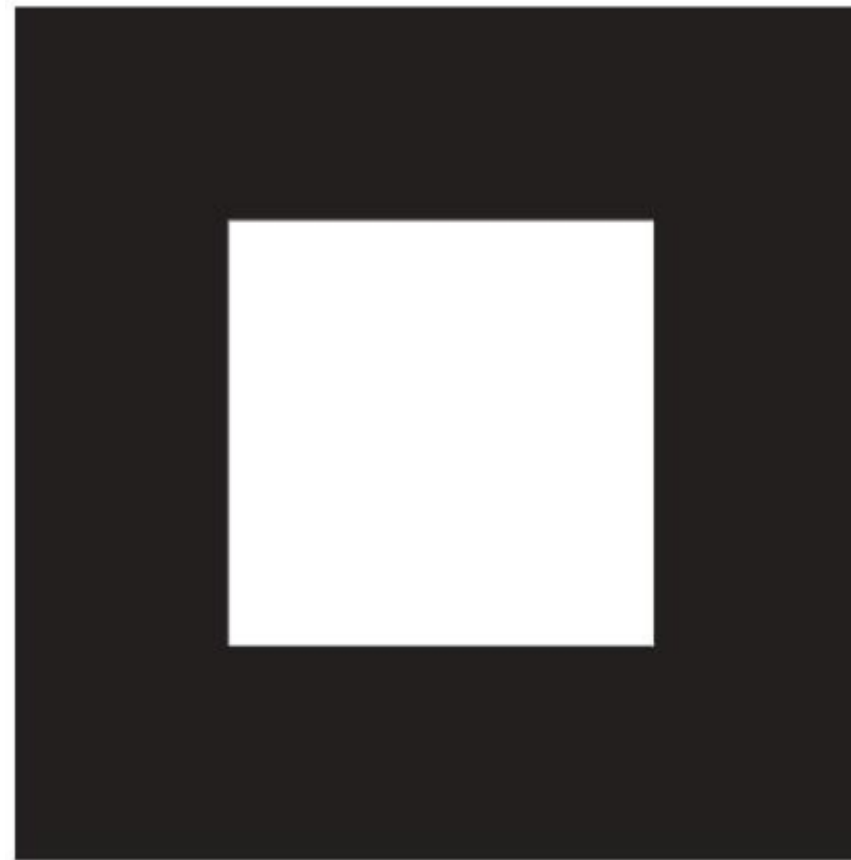
3. Dự đoán bù chuyển động



3. Dự đoán bù chuyển động



Hình ảnh hiện tại



Hình ảnh dự đoán

3.1.Ước lượng chuyển động

- ▶ Một cách tiếp cận là phân chia từng hình ảnh trong chuỗi video thành các đối tượng riêng lẻ.
- ▶ Sau đó có thể tìm kiếm cùng một hình ảnh dự đoán trước đó và ước tính chuyển động của đối tượng giữa các hình ảnh.
- ▶ Mặc dù cách này có thể mang lại hiệu suất tuyệt vời, nhưng có một số khó khăn liên quan đến cách tiếp cận này.

3.1.Ước lượng chuyển động

- ▶ Đầu tiên là khó khăn trong việc tìm các đối tượng riêng lẻ trong một hình ảnh.
- ▶ Quá trình này, được gọi là **phân đoạn** (**segmentation**).
- ▶ Các kỹ thuật phân đoạn đáng tin cậy vẫn chưa được tìm thấy.

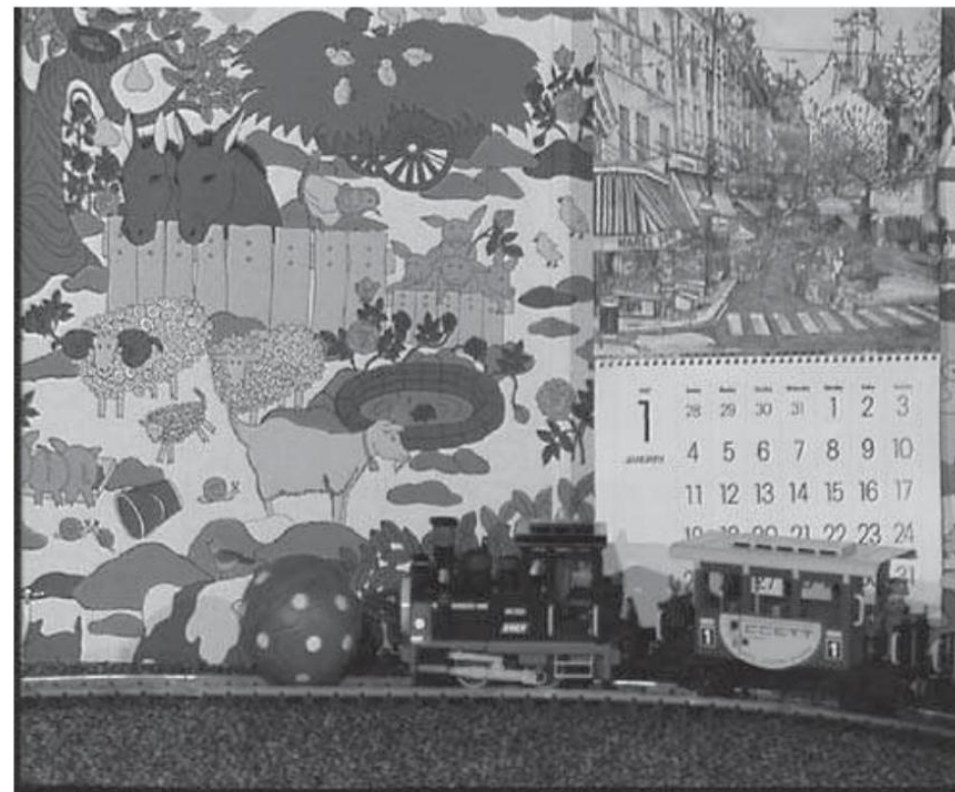
3.1.Ước lượng chuyển động

- ▶ Vấn đề thứ hai là các đối tượng không di chuyển theo cách tuần tự tịnh tiến.
- ▶ Do đó, rất có thể sẽ có sự khác biệt giữa đối tượng khi nó xuất hiện trong hình ảnh hiện tại và cách nó xuất hiện trong hình ảnh dự đoán khiến quá trình khớp trở nên khó khăn hơn nhiều.

3.1.Ước lượng chuyển động

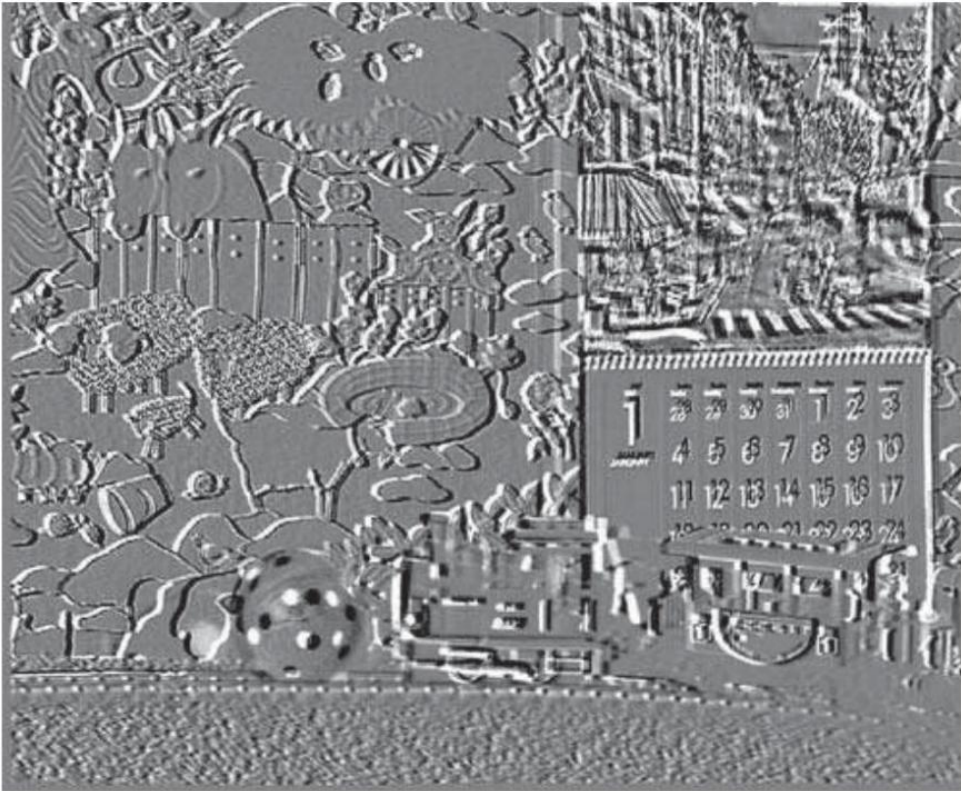
- ▶ Cách tiếp cận thông thường để ước tính chuyển động là chia hình ảnh hiện tại cần dự đoán thành một số khối.
- ▶ Hình ảnh dự đoán, đã được mã hóa, sau đó được tìm kiếm khối phù hợp nhất với từng khối này trong hình ảnh hiện tại.

3.1.Ước lượng chuyển động



Hình ảnh dùng để dự đoán (frame 1) Hình ảnh cần dự đoán (frame 5)

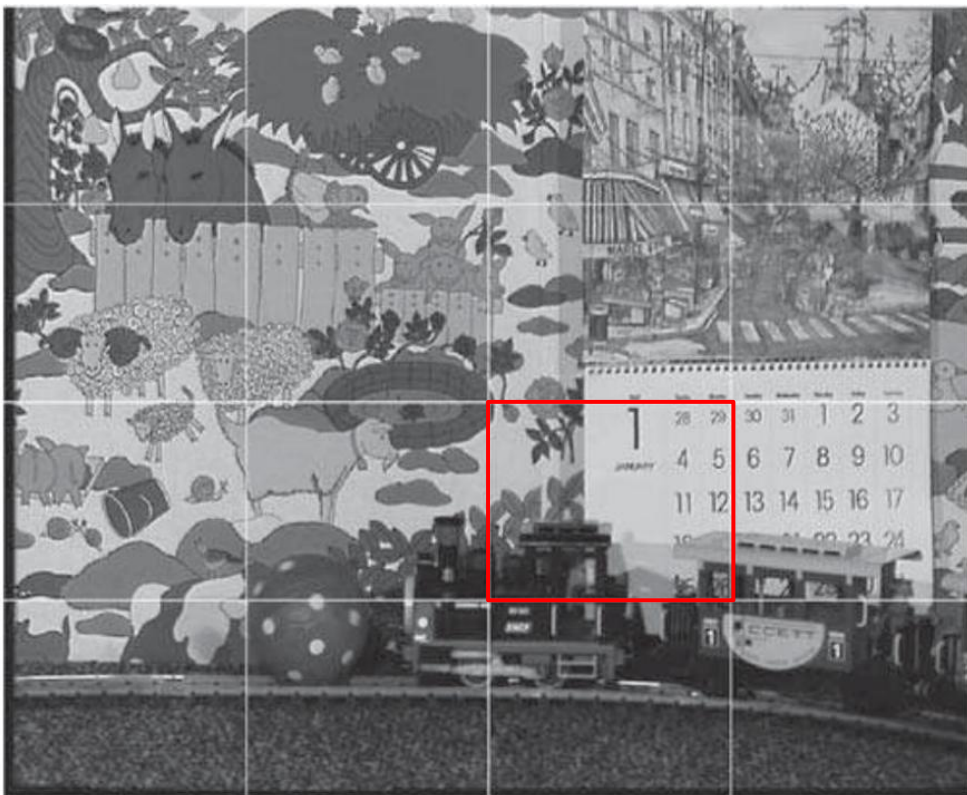
3.1.Ước lượng chuyển động



- Mục đích của ước tính chuyển động là tìm một khối trong bức ảnh dùng để dự đoán (frame 1) khớp nhất với mỗi khối trong bức ảnh hiện tại (frame 5).

Sự khác nhau giữa 2 hình trên

3.1.Ước lượng chuyển động



Frame 1 chia thành các khối



Frame 5 chia thành các khối

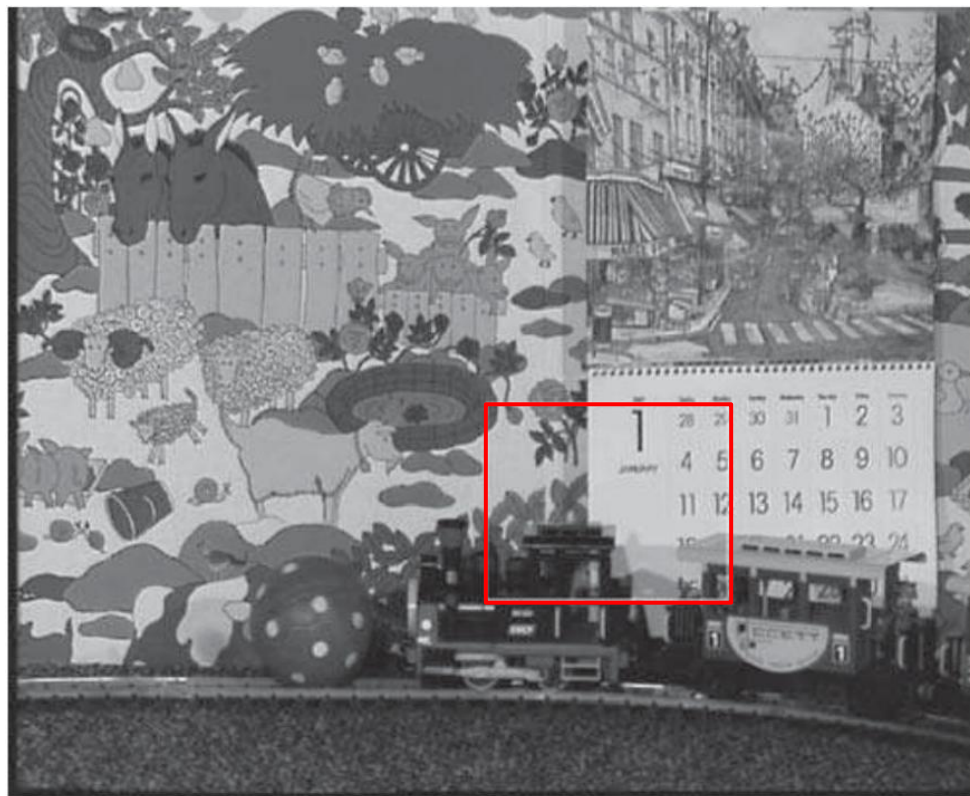
3.1.Ước lượng chuyển động



Sự khác nhau giữa
2 khối

- ▶ Quá trình ước tính chuyển động bao gồm việc tìm khối có cùng kích thước trong hình ảnh dự đoán phù hợp nhất với khối trong hình ảnh hiện tại.
- ▶ Một phương pháp tính toán đơn giản sẽ chỉ ra rằng khối phù hợp nhất sẽ được tìm thấy cách khối đồng vị trí bốn pixel về bên trái và cách khối đồng vị trí một pixel trong hình ảnh dự đoán.

3.1.Ước lượng chuyển động



Khôi khớp nhất trong frame 1



Sự khác nhau giữa
2 khối (mới)



Sự khác nhau giữa
2 khối (cũ)

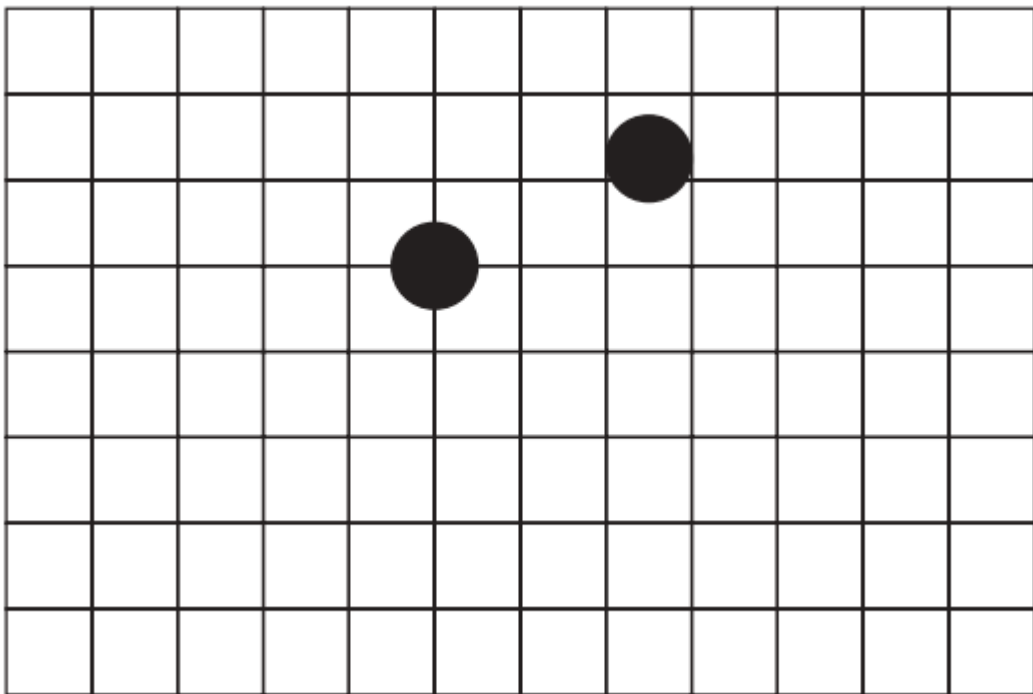
3.1.Ước lượng chuyển động

- ▶ Sự khác biệt tuyệt đối trung bình giảm từ 25,5 xuống 18,4 (giảm 30%). Lưu ý rằng mặc dù hình nền được dự đoán khá tốt, nhưng các con số trên lịch và tàu hỏa lại kém dự đoán hơn nhiều.
- ▶ Đây là do thực tế là chuyển động của các vật thể là khác nhau. Phương pháp ước tính chuyển động được sử dụng không cho phép ước tính hai hoặc nhiều chuyển động khác nhau trong cùng một khối.

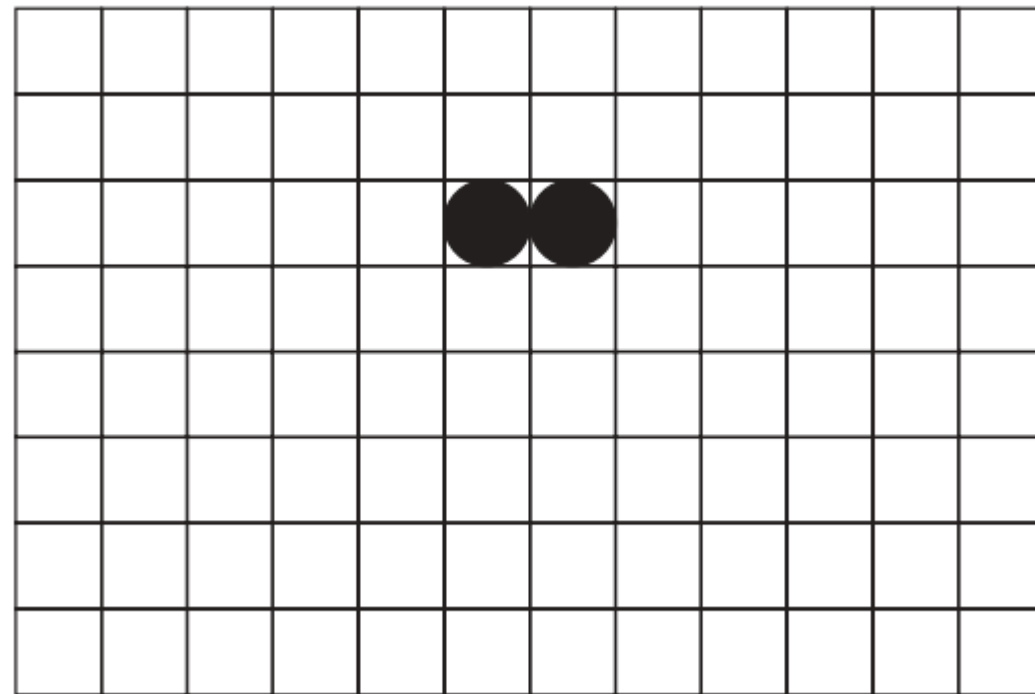
3.1.Ước lượng chuyển động

- Dự đoán bù chuyển động bao gồm hai phần: ước lượng chuyển động (**motion estimation**) (chỉ được thực hiện tại bộ mã hóa) và bù chuyển động (**motion compensation**) (cần được thực hiện tại cả bộ mã hóa và bộ giải mã).

3.1. Ước lượng chuyển động

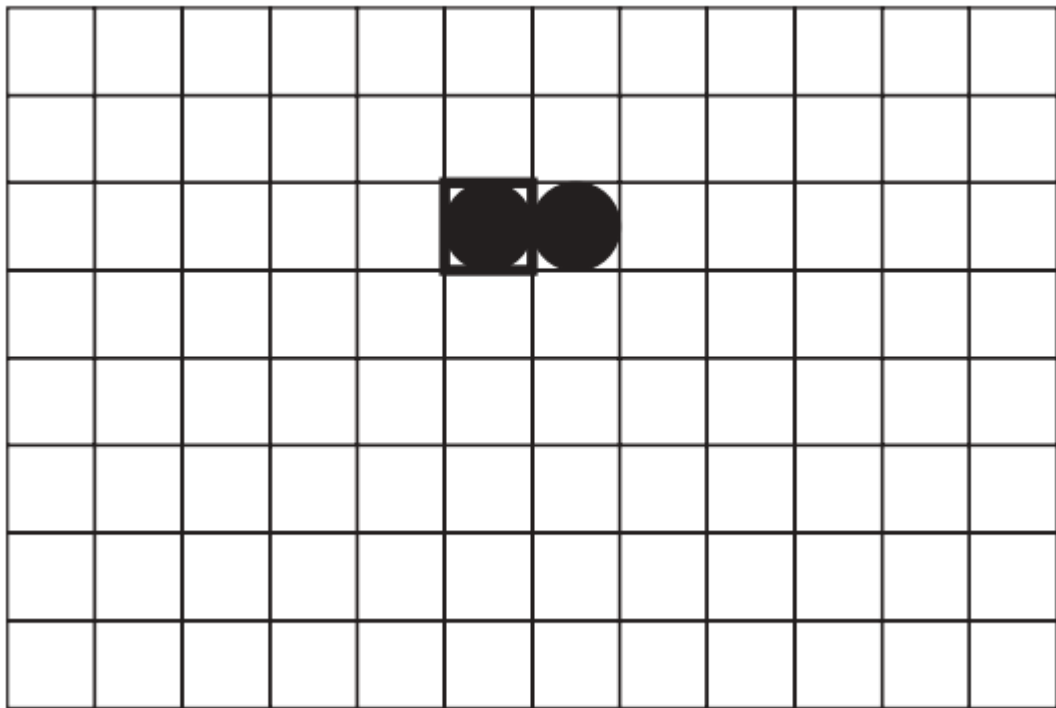


Hình ảnh dùng để dự đoán

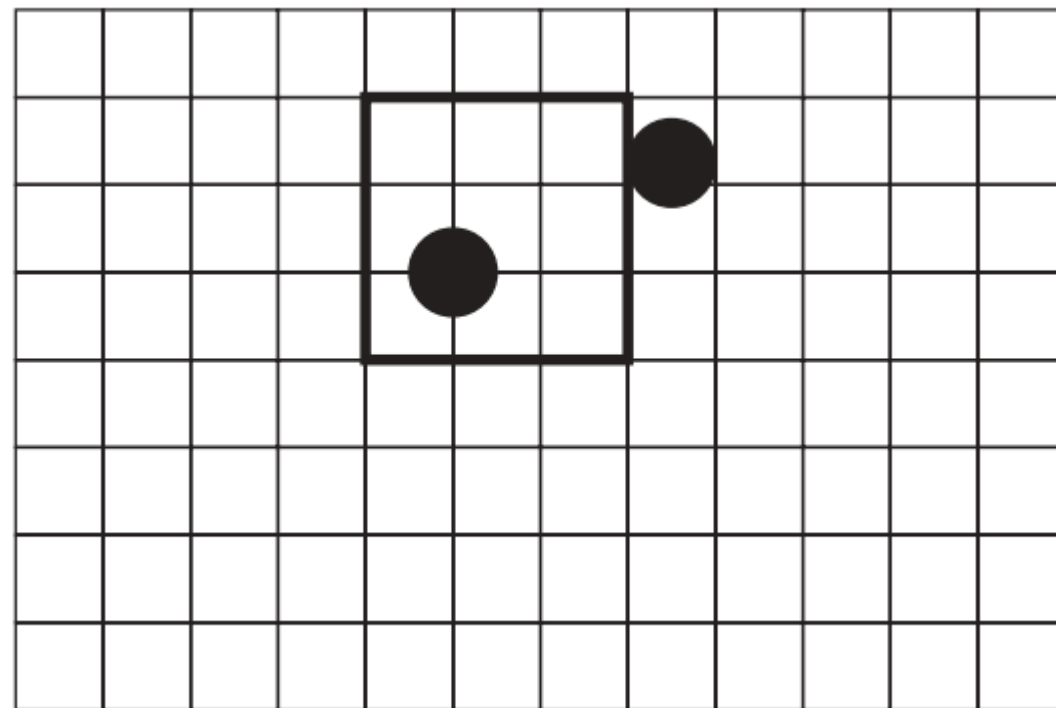


Hình ảnh cần dự đoán

3.1.Ước lượng chuyển động

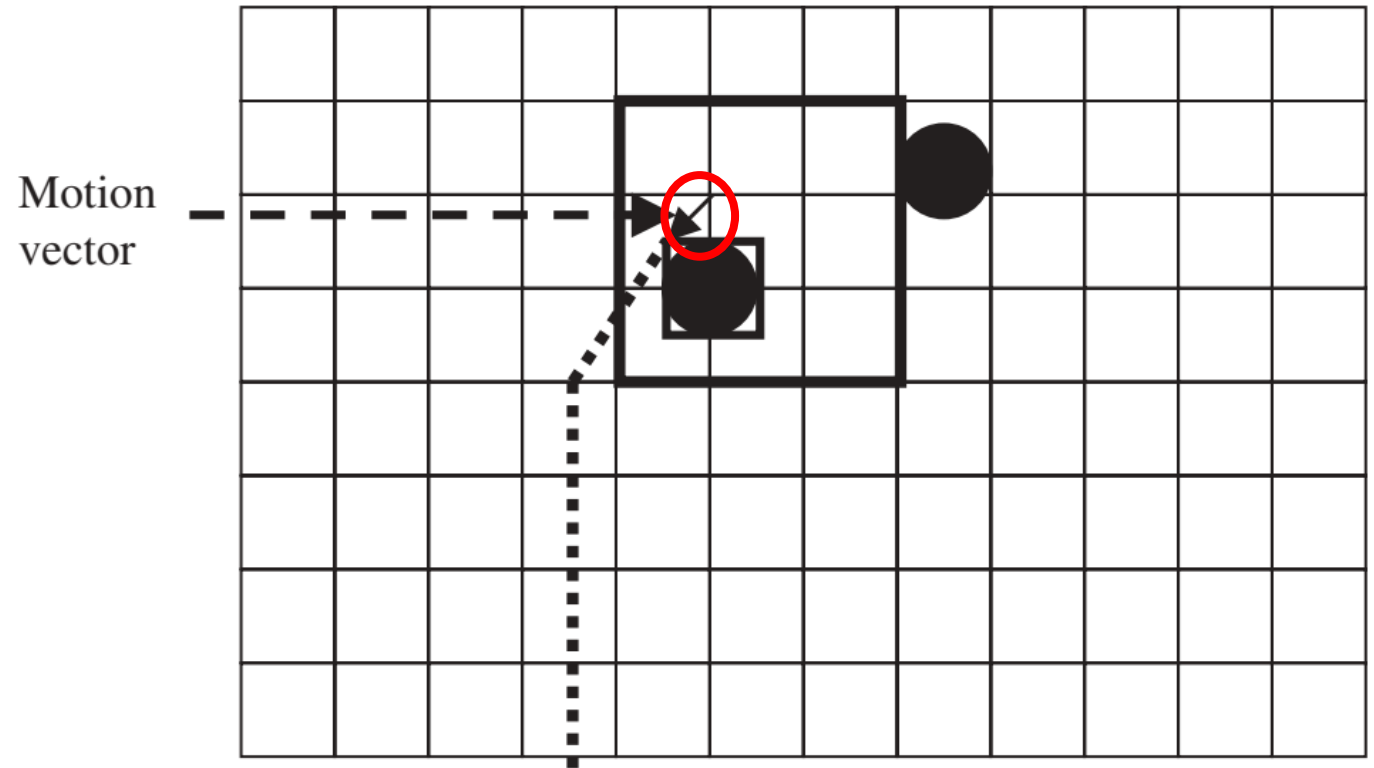
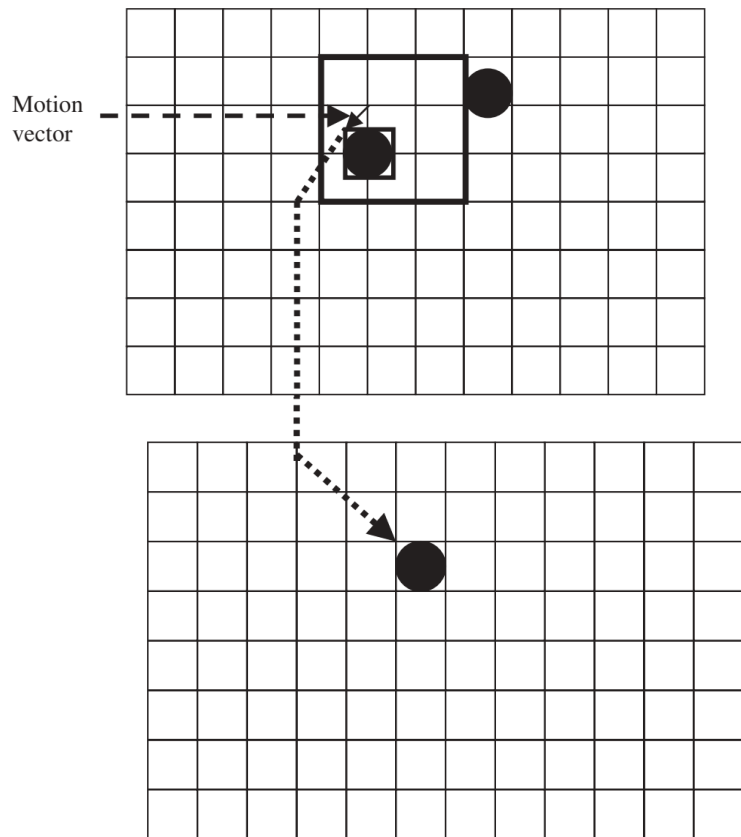


Khối cần tìm



Vùng tìm kiếm trong hình
dùng để dự đoán

3.1.Ước lượng chuyển động



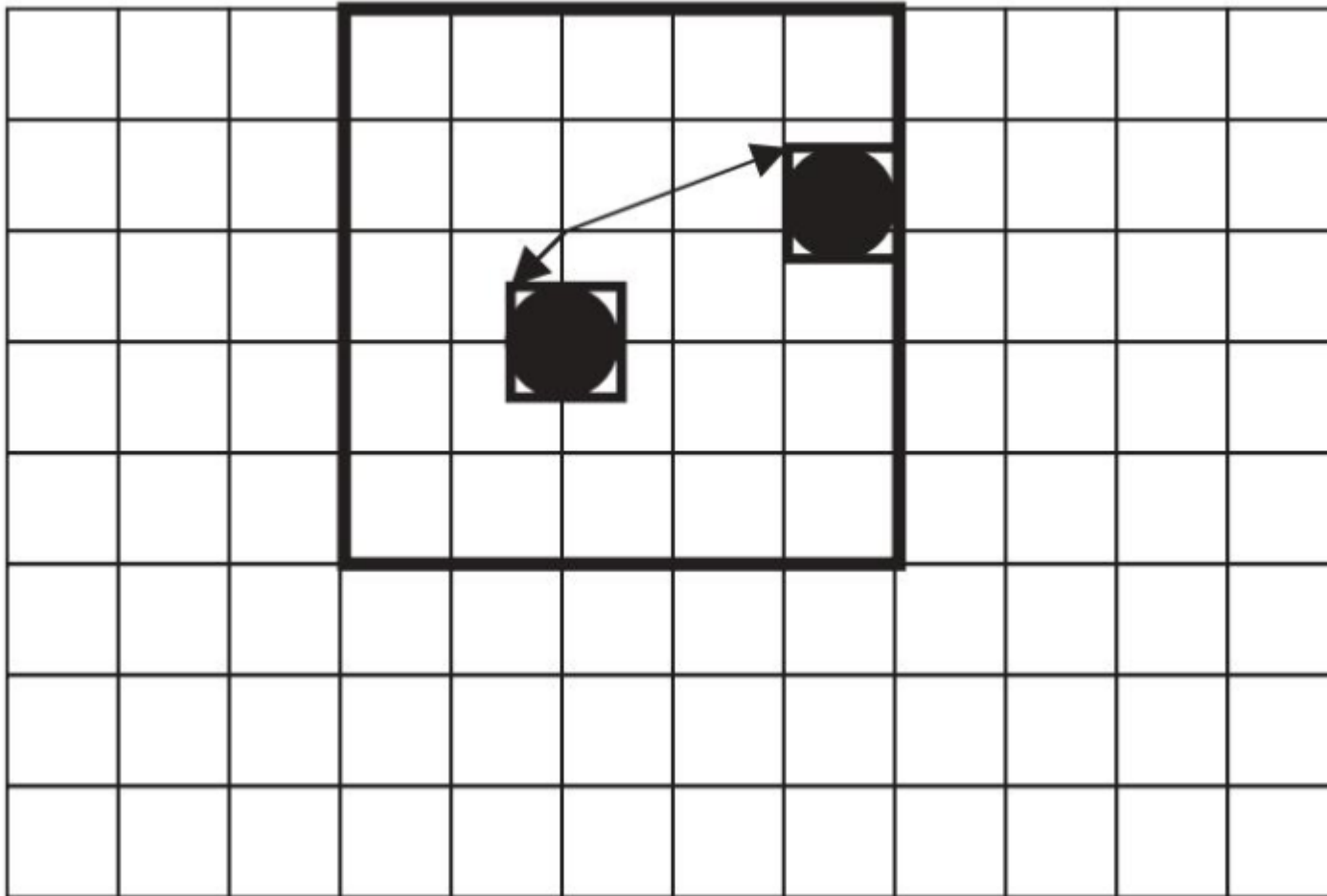
3.1.Ước lượng chuyển động

- ▶ Vị trí của khối trong hình ảnh dự đoán được truyền đến bộ giải mã vì bộ giải mã không có quyền truy cập vào cùng thông tin như bộ mã hóa (ví dụ: phiên bản gốc của hình ảnh hiện tại và hình ảnh dự đoán) và do đó không thể tính toán chính vị trí đó.
- ▶ Thông tin được truyền đến bộ giải mã được gọi là vectơ chuyển động.

3.1.Ước lượng chuyển động

- ▶ Trong trường hợp này, vector chuyển động được hiển thị trong hình ảnh dự đoán bằng một mũi tên liền.
- ▶ Nó đánh dấu độ dịch chuyển từ vị trí góc trên cùng bên trái của khối trong hình ảnh hiện tại đang được dự đoán đến vị trí góc trên cùng bên trái của khối dự đoán đã chọn trong hình ảnh dự đoán.

3.1.Ước lượng chuyển động



- Nếu vùng tìm kiếm rộng hơn, ta có kết quả như hình bên

3.1.Ước lượng chuyển động

- ▶ Trong nén video, mục đích là tìm ra sự khớp tốt, không nhất thiết phải ước tính chuyển động chính xác.
- ▶ Nhìn chung, việc xác định khối phù hợp không dễ dàng vì khối phù hợp nhất thường không giống với khối trong hình ảnh hiện tại.

3.1.Ước lượng chuyển động

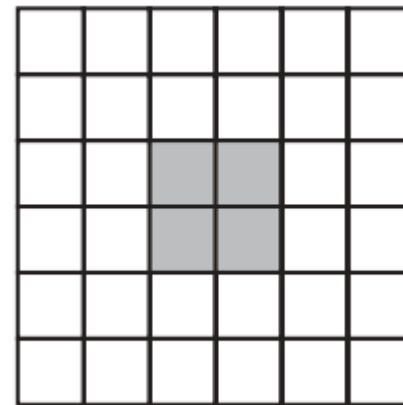
- ▶ Hai tiêu chí phổ biến nhất là tổng chênh lệch tuyệt đối và tổng chênh lệch bình phương giữa khối dự đoán hiện tại và khối ứng viên phù hợp.
- ▶ Cách tìm kiếm là so sánh khối hiện tại với mọi khối có cùng kích thước trong vùng tìm kiếm.

3.1.Ước lượng chuyển động

- **Ví dụ:** có tổng cộng bao nhiêu khối cần so sánh với khối gốc trong vùng tìm kiếm?



Khối gốc



Vùng tìm kiếm (± 2 pixel)

3.1.Ước lượng chuyển động

- ▶ Đối với một vùng tìm kiếm $\pm N$ pixel theo cả chiều ngang và chiều dọc, tổng số vị trí tìm kiếm cần xem xét là:
$$(2N + 1)(2N + 1)$$
- ▶ Đối với một vùng tìm kiếm lớn, đây có thể là một số lượng rất lớn các vị trí tìm kiếm.

3.1.Ước lượng chuyển động

- ▶ **Ví dụ:** Ước tính chuyển động tìm kiếm đầy đủ dựa trên khối với vùng tìm kiếm ± 16 pixel được sử dụng để hình thành dự đoán về hình ảnh hiện tại từ hình ảnh dự đoán được mã hóa trước đó. Kích thước khối là 16×16 pixel. Mỗi hình ảnh có độ phân giải 704×480 pixel với tốc độ hình ảnh là 30 hình ảnh/giây.

3.1.Ước lượng chuyển động

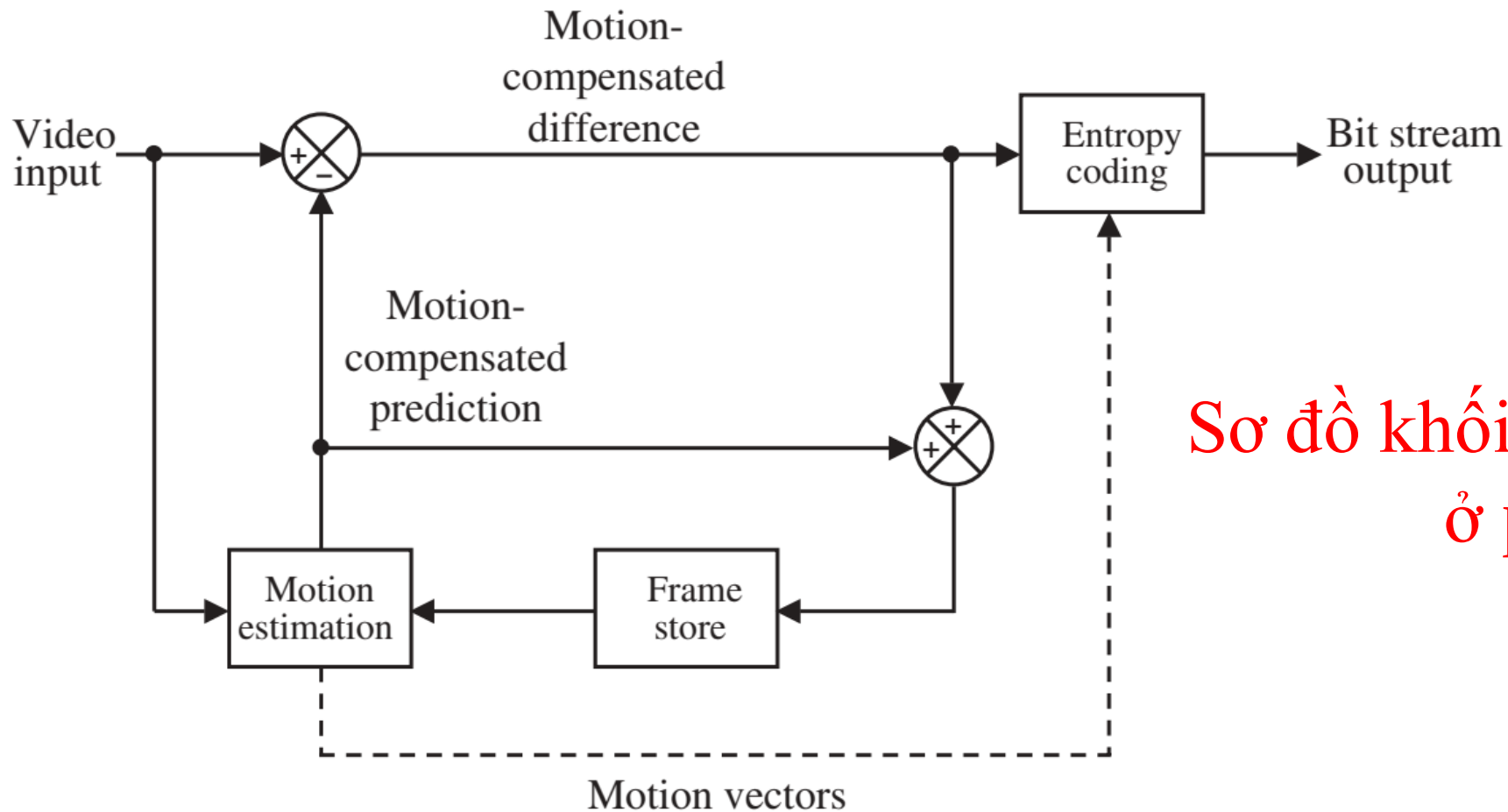
► Tính:

- Số lượng vị trí cần tìm kiếm cho mỗi khối.
- Số lượng pixel cần so sánh cho mỗi khối.
- Số lượng pixel cần so sánh trên mỗi hình ảnh.
- Số lượng pixel cần so sánh mỗi giây.

3.1.Ước lượng chuyển động

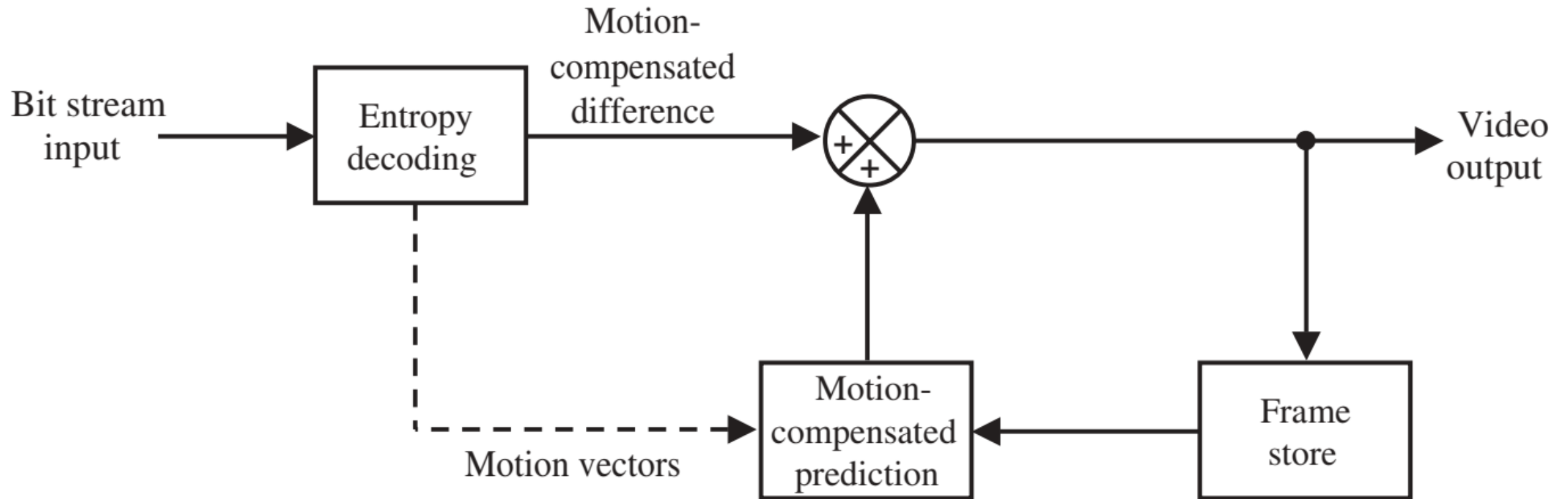
- ▶ Yêu cầu tính toán của ước tính chuyển động thực sự lớn.
- ▶ May mắn thay, phần cứng xử lý tín hiệu số chuyên dụng có khả năng hoạt động ở các tốc độ này.
- ▶ Hầu hết các tiêu chuẩn mã hóa video sử dụng kích thước khối là 16×16 pixel để dự đoán bù chuyển động.

3.1. Ước lượng chuyển động



Sơ đồ khối bù chuyển động
ở phía phát

3.1.Ước lượng chuyển động



Sơ đồ khối bù chuyển động ở phía thu

3.2. Dự đoán bù chuyển động với độ chính xác subpixel

- ▶ Dự đoán bù chuyển động theo độ chính xác của subpixel được hỗ trợ trong nhiều tiêu chuẩn mã hóa video bao gồm cả các tiêu chuẩn được sử dụng cho các dịch vụ truyền hình kỹ thuật số.
- ▶ Điều này đạt được bằng cách ước tính giá trị của pixel tại các subpixel bằng cách sử dụng nội suy song tuyến tính từ các giá trị pixel đã biết.

3.2. Dự đoán bù chuyển động với độ chính xác subpixel

- **Ví dụ:** cho khối dữ liệu hình ảnh 6×6 và khối trung tâm 4×4 như hình vẽ.
- Tính giá trị của khối 4×4 nằm cách nửa pixel về bên phải của khối trung tâm.

99	93	70	71	96	96
100	82	70	88	97	87
71	96	78	77	98	92
54	67	71	93	98	84
109	55	50	69	92	92
164	110	50	46	73	83

3.2. Dự đoán bù chuyển động với độ chính xác subpixel

- Tính giá trị của khối 4×4 nằm cách nửa pixel về bên dưới của khối trung tâm.
- Tính giá trị của khối 4×4 nằm cách nửa pixel về bên trái và cách nửa pixel về bên trên của khối trung tâm.

(Chạy file Matlab b3_07)

3.2. Dự đoán bù chuyển động với độ chính xác subpixel

- ▶ Có thể thực hiện dự đoán bù chuyển động với độ chính xác dưới một nửa pixel.
- ▶ Ví dụ, có thể nội suy giữa các điểm nửa pixel để tính toán một phần tư điểm pixel.
- ▶ Tuy nhiên, đối với các dịch vụ truyền hình kỹ thuật số dựa trên MPEG-2, dự đoán bù chuyển động bị giới hạn ở độ chính xác tối đa là một nửa pixel.

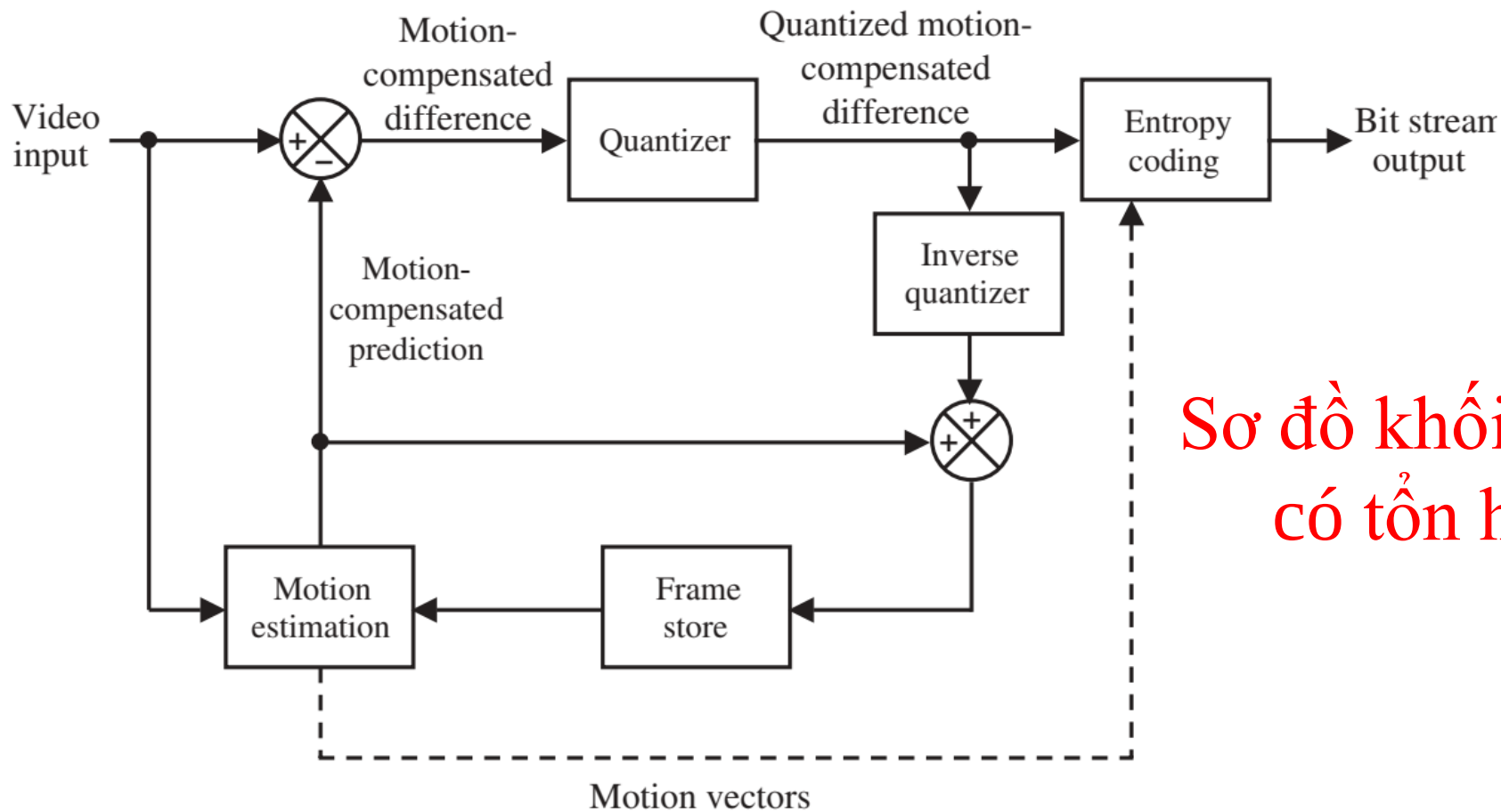
4. Lượng tử hóa

- ▶ Tất cả các kỹ thuật trên đều không làm mất dữ liệu (**lossless**).
- ▶ Điều này có nghĩa là tín hiệu video được giải mã giống hệt về mọi mặt với tín hiệu đi vào bộ mã hóa.
- ▶ Mặc dù điều này đảm bảo rằng tín hiệu nhận được có chất lượng cao nhất có thể tuy nhiên lượng nén có thể đạt được là hạn chế.

4. Lượng tử hóa

- ▶ May mắn thay, có thể nói lỏng ràng buộc này để đạt được mức nén cao hơn trong khi vẫn duy trì chất lượng dịch vụ video.
- ▶ Điều này đạt được bằng cách loại bỏ thông tin không quan trọng đối với chất lượng chủ quan của video.
- ▶ Việc xác định thông tin nào là quan trọng và thông tin nào không là một nhiệm vụ khá khó khăn, kỹ thuật loại bỏ thông tin dễ hiểu hơn nhiều.

4. Lượng tử hóa

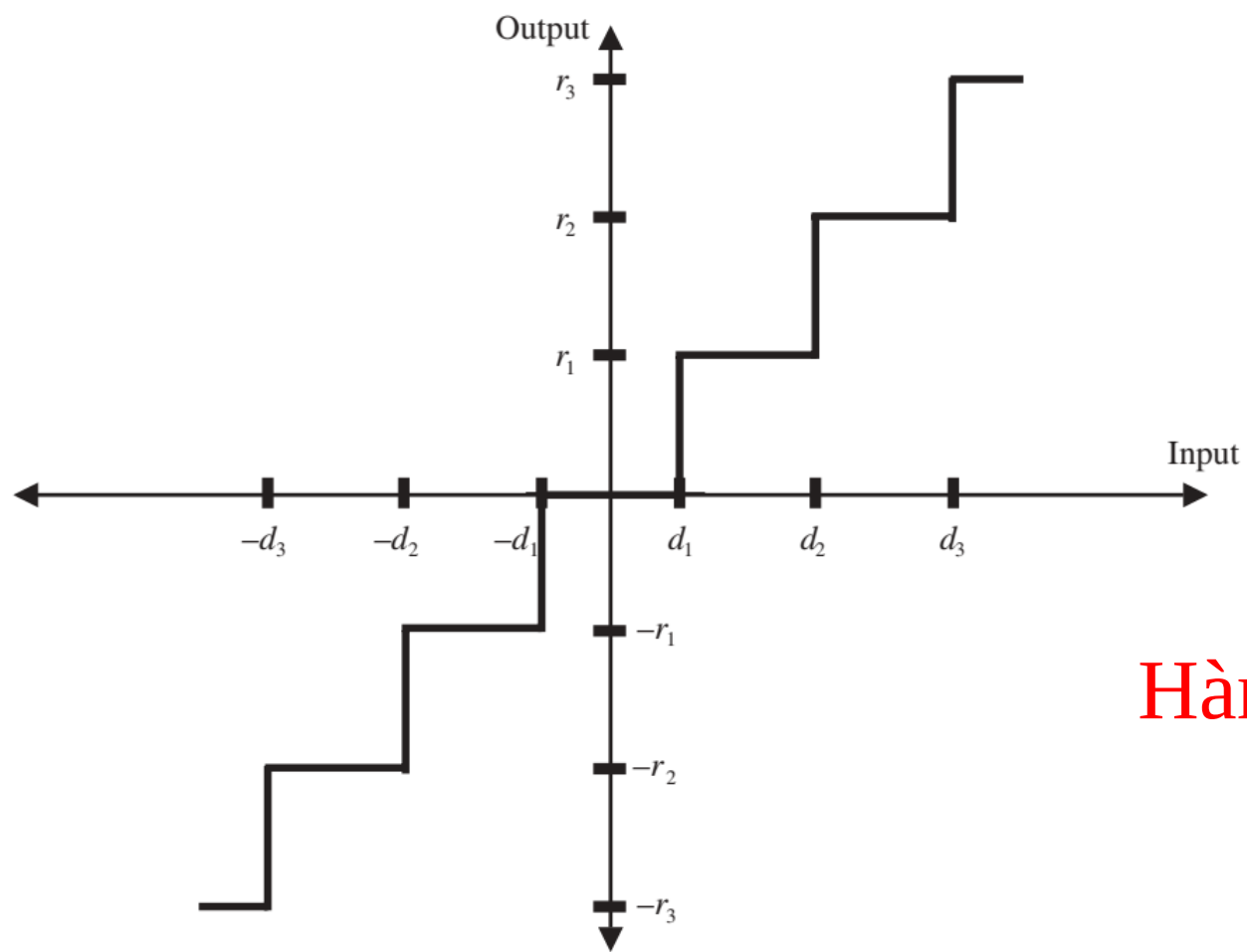


Sơ đồ khối bù chuyển động
có tổn hao ở phía phát

4. Lượng tử hóa

- ▶ Thiết kế bộ lượng tử hóa đã được nghiên cứu rộng rãi.
- ▶ Bộ lượng tử hóa được xác định theo vị trí của các mức quyết định và tái tạo của nó.
- ▶ Hình dưới thể hiện cho bộ lượng tử hóa tuyến tính trong đó khoảng cách giữa các mức quyết định là hằng số.

4. Lượng tử hóa



Hàm truyền của bộ lượng tử hóa

4. Lượng tử hóa

Lower decision level	Upper decision level	Reconstruction level
$-\infty$	$-d_3$	$-r_3$
$-d_3$	$-d_2$	$-r_2$
$-d_2$	$-d_1$	$-r_1$
$-d_1$	$+d_1$	0
$+d_1$	$+d_2$	$+r_1$
$+d_2$	$+d_3$	$+r_2$
$+d_3$	$+\infty$	$+r_3$

Cách thức hoạt động của bộ lượng tử hóa

4. Lượng tử hóa

- ▶ Bộ lượng tử hóa như vậy hiệu quả nhất đối với tín hiệu đầu vào trong đó tất cả các giá trị đều có khả năng xảy ra như nhau.
- ▶ Đối với một bộ lượng tử tuyến tính, mức tái tạo thường là mức trung bình của hai mức quyết định bao quanh nó.

4. Lượng tử hóa

145	162	199	155	112
149	151	153	154	162
92	85	88	187	212

- **Ví dụ:** Cho ma trận điểm ảnh như sau:
- Tính toán sự khác biệt dự đoán khi sử dụng dự đoán một chiều bằng pixel nằm bên trái pixel cần dự đoán. Giả sử rằng bất kỳ pixel nằm ngoài hình ảnh đều có giá trị 128.
 - Sự khác biệt dự đoán được lượng tử hóa bằng cách sử dụng bộ lượng tử hóa tuyến tính với bước lượng tử là 8. Tính toán dữ liệu được tái tạo sau khi dự đoán và lượng tử hóa.

4. Lượng tử hóa

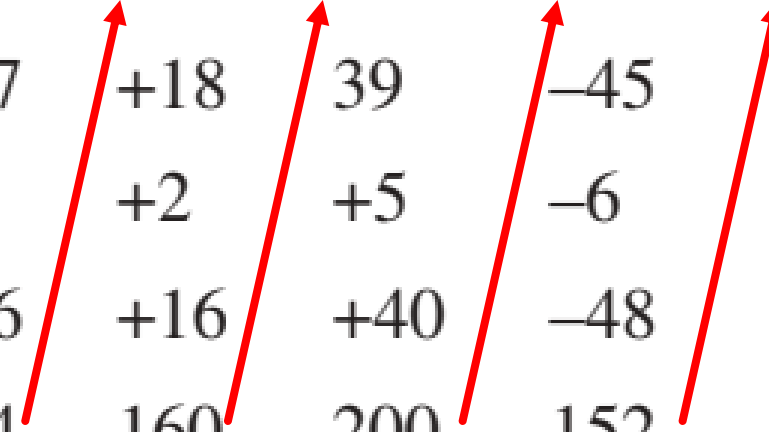
- ▶ Các bước tính toán có thể được tóm tắt như sau:
 - **Prediction**: là giá trị dự đoán.
 - **Prediction difference**: là sự khác biệt dự đoán giữa giá trị của pixel hiện tại và giá trị dự đoán.
 - **Quantizer prediction difference**: là chênh lệch dự đoán chia cho bước lượng tử và được làm tròn đến số nguyên gần nhất.

4. Lượng tử hóa

- **Reconstructed prediction difference**: là sự khác biệt dự đoán được lượng tử hóa nhân với bước lượng tử.
- Cuối cùng, **reconstructed pixel** là giá trị dự đoán cộng với sự khác biệt dự đoán được tái tạo.
- Giá trị được tái tạo này được sử dụng làm dự đoán cho pixel tiếp theo.

4. Lượng tử hóa

Current pixel	145	162	199	155	112
Prediction	128	144	160	200	152
Prediction difference	+17	+18	39	-45	-40
Quantized prediction difference	+2	+2	+5	-6	-5
Reconstructed prediction difference	+16	+16	+40	-48	-40
Reconstructed pixel	144	160	200	152	112



(Chạy file Matlab b3_08)



Thank
You

FOR
LISTENING